

SUPFile

Plateforme de stockage cloud nouvelle génération



Documentation Technique

Version 1.0

Projet : 4PROJ Développement – SUPINFO

Mandant : SUPFile

Équipe : Luka Karkashadze, Bastien Marcheron,
Béatrice Beavogui, Théo Monel

Date : Mai 2026

Dépôt : github.com/HamerTails/4_Proj_Drive

Démo : supfile.hackthehydra.com

Préface

Ce document constitue la documentation technique exhaustive du projet **SUPFile**, plateforme de stockage cloud développée dans le cadre du module 4PROJ à SUPINFO. Il couvre l'intégralité du cycle de développement, depuis la conception jusqu'au déploiement en production, en passant par les choix architecturaux, l'implémentation détaillée des trois clients (API, web, mobile), la stratégie de sécurité, et la procédure de mise en production sur infrastructure VPS.

Le projet a pour objectif de proposer une alternative open source crédible à des services tels que Google Drive et Dropbox, avec une infrastructure souveraine et un quota généreux de 30 Go par utilisateur en version commerciale. La version V1 livrée couvre l'intégralité du cahier des charges fonctionnel, à l'exception d'une limitation documentée concernant la navigation interne des dossiers partagés, fonctionnalité prévue dans la roadmap V2.

L'instance de démonstration est accessible publiquement à l'adresse <https://supfile.hackthehydra.com>, déployée sur un VPS Alpine Linux avec Docker Compose et reverse proxy Caddy assurant le HTTPS via Let's Encrypt.

Public visé : ce document s'adresse principalement aux décideurs SUPFile (DSI, Product Owner, Sponsors), aux équipes techniques chargées de la reprise et de la TMA, ainsi qu'à toute personne souhaitant comprendre, étendre ou auditer la solution.

Conventions de lecture :

- Les blocs de code sont en police monospace avec coloration syntaxique.
- Les chemins de fichiers du dépôt sont notés en `\texttt`.
- Les références externes sont en lien hypertexte bleu.
- Les diagrammes UML sont fournis en TikZ ou référencés dans `docs/UML.md` pour le rendu Mermaid.

Table des matières

Préface	2
Liste des figures	11
Liste des tableaux	12
I Présentation du projet	13
1 Contexte et objectifs	14
1.1 Contexte général	14
1.2 Objectifs du projet	14
1.2.1 Objectifs fonctionnels	14
1.2.2 Objectifs techniques	15
1.2.3 Contraintes du projet	15
1.3 Positionnement et différenciation	15
1.4 Cycle de livraison	16
2 Méthodologie	17
2.1 Approche projet	17
2.2 Découpage en sprints	17
2.3 Outils de collaboration	17
2.4 Pratiques de développement	18
2.4.1 Convention de commits	18
2.4.2 Tests et qualité	18
2.5 Défis techniques rencontrés	19
2.5.1 Streaming audio et vidéo	19
2.5.2 Génération de ZIP à la volée	19
2.5.3 Arborescence récursive	19
2.5.4 Iframe cross-origin pour preview PDF	20
3 Présentation de l'équipe	21
3.1 Composition	21
3.2 Statistiques de contribution	22
3.3 Communication et processus	22
4 Conformité au cahier des charges	23
4.1 Vue d'ensemble du barème	23
4.2 Tableau de conformité fonctionnelle	23
4.3 Bonus implémentés	25

4.4	Conformité documentaire	27
4.5	Conformité technique	27
4.5.1	Architecture trois tiers (sujet §2.3.1)	27
4.5.2	Containérisation (sujet §2.3.2)	27
4.5.3	Sécurité (sujet §4, malus secrets)	28
5	Modèle économique et budget	29
5.1	Coûts de développement (V1)	29
5.1.1	Ressources humaines	29
5.1.2	Infrastructure de développement	29
5.2	Coûts d'exploitation mensuels (V1)	29
5.3	Modèle économique cible (V2)	30
5.3.1	Plans tarifaires	30
5.3.2	Projection financière 3 ans	30
5.3.3	Sources de revenus complémentaires (post-V2)	31
5.3.4	Avantage compétitif et différenciation	31
5.4	Planning du projet (diagramme de Gantt)	31
II	Architecture et choix techniques	32
6	Architecture globale	33
6.1	Vue d'ensemble	33
6.2	Découpage des couches	33
6.2.1	Couche présentation	33
6.2.2	Couche logique métier	34
6.2.3	Couche persistance	34
6.3	Flux de données principaux	34
6.3.1	Connexion utilisateur	34
6.3.2	Upload d'un fichier	35
6.3.3	Prévisualisation d'un fichier	35
6.3.4	Partage public	35
7	Choix technologiques	36
7.1	Vue d'ensemble de la stack	36
7.2	Justification des choix backend	36
7.2.1	Node.js + Express	36
7.2.2	PostgreSQL 15	37
7.2.3	JWT pour l'authentification	37
7.3	Justification des choix frontend web	38
7.3.1	React + Vite	38
7.3.2	react-router-dom	38
7.3.3	recharts	38

7.4	Justification des choix mobile	38
7.4.1	React Native + Expo	38
7.4.2	Expo SDK 54	39
7.4.3	expo-router 6	39
7.4.4	AsyncStorage	40
7.5	Justification des choix DevOps	40
7.5.1	Docker Compose v2	40
7.5.2	Caddy 2 comme reverse proxy	40
7.5.3	Alpine Linux comme OS de production	41
8	Modèle de données	42
8.1	Vue d'ensemble du schéma	42
8.1.1	Diagramme entité-relation	42
8.2	Détail des tables	42
8.2.1	Table users	42
8.2.2	Table nodes	43
8.2.3	Table shares	44
8.2.4	Table internal_shares	44
8.3	Indexes secondaires	44
III	Implémentation	46
9	API REST	47
9.1	Organisation des routes	47
9.2	Middlewares globaux	47
9.3	Middleware d'authentification	47
9.4	Catalogue des endpoints	48
9.4.1	Authentification	48
9.4.2	Nodes (fichiers et dossiers)	49
9.4.3	Fichiers (upload/download/preview)	50
9.4.4	Stockage et quotas	50
9.4.5	Partage	51
9.4.6	Recherche	51
9.4.7	Corbeille	51
9.4.8	Utilisateurs	52
10	Frontend Web	53
10.1	Structure de l'application	53
10.2	Service HTTP centralisé	53
10.3	Gestion du thème	53
10.4	Drag and drop	54
10.5	Modales de prévisualisation	54

10.6	Sidebar redimensionnable	54
10.7	Toasts et notifications	54
10.8	Skeletons et loaders	55
10.9	Wireframes des écrans clés	55
10.9.1	Login	55
10.9.2	Dashboard accueil	56
10.9.3	Navigateur de fichiers	56
10.9.4	Modale de partage	57
10.10	Code d'exemple : composant Toast	57
11	Application Mobile	58
11.1	Vue d'ensemble	58
11.2	Structure des écrans	58
11.3	Différences notables avec le web	58
11.4	Gestion du token et navigation	58
11.5	ThemeContext et persistance du thème	59
11.6	Permissions runtime	59
11.7	Streaming audio/vidéo mobile	60
11.8	FolderPicker pour le déplacement	60
11.9	Wireframes mobile	61
11.9.1	Écran d'accueil et login	61
11.9.2	Navigateur de fichiers mobile	61
11.10	Différences UX web vs mobile	62
IV	Qualité et déploiement	63
12	Sécurité	64
12.1	Authentification et autorisation	64
12.1.1	Hash des mots de passe	64
12.1.2	JWT	64
12.1.3	OAuth Google	64
12.1.4	Vérification d'ownership	64
12.2	Protection contre les attaques courantes	64
12.2.1	Injection SQL	64
12.2.2	XSS	65
12.2.3	CSRF	65
12.2.4	Brute force	65
12.2.5	Iframe clickjacking	65
12.3	Gestion des secrets	65
12.3.1	Principes appliqués	65
12.3.2	Inventaire des secrets de l'application	66
12.3.3	Procédure de rotation	66

12.3.4	Audit de présence de secrets	66
12.3.5	Cas particulier : <code>docker-compose.yml</code> en développement	67
12.3.6	Conformité au sujet	67
13	Tests et qualité du code	68
13.1	Conventions de code	68
13.1.1	Backend	68
13.1.2	Frontend web	68
13.1.3	Frontend mobile	68
13.2	Validation des entrées	68
13.3	Stratégie de tests	69
13.4	Matrice tests × fonctionnalités	69
13.5	Plan de tests E2E	71
13.5.1	Test E2E web (Claude Chrome – 23 mai 2026)	71
13.5.2	Test mobile Android	71
13.6	Tests fonctionnels	72
13.6.1	Tests d'API automatisés	72
13.6.2	Tests UI via Claude Chrome	72
14	Analyse de risques	73
14.1	Matrice de risques	73
14.2	Risques techniques	73
14.3	Risques sécurité	74
14.4	Risques projet	74
14.5	Risques opérationnels	75
14.6	Plan de continuité de service (PCA)	75
15	Déploiement	76
15.1	Pré-requis	76
15.1.1	Environnement de développement	76
15.1.2	Environnement de production	77
15.1.3	Vérifications préalables	77
15.2	Stack de production	77
15.3	Procédure d'installation	77
15.4	Configuration DNS Cloudflare	78
15.5	HTTPS et certificats	78
15.6	Sauvegarde et restauration	78
16	Charte graphique	80
16.1	Palette de couleurs	80
16.1.1	Mode sombre	80
16.2	Typographie	80
16.3	Composants visuels	80

16.4	Logo	81
16.4.1	Variantes visuelles du logo	81
16.5	Palette visualisée	82
16.6	Système d'espacement	82
16.7	Mockups de composants	82
16.7.1	Boutons	82
16.7.2	Cards et inputs	83
16.7.3	Toasts	83
16.8	Iconographie	83
V	Perspectives et annexes	84
17	Limitations V1 et roadmap V2	85
17.1	Limitations connues V1	85
17.2	Roadmap V2	85
17.2.1	Trimestre 1	85
17.2.2	Trimestre 2	85
17.2.3	Trimestre 3-4	85
18	Cas d'utilisation et diagrammes UML	87
18.1	Diagramme de cas d'utilisation	87
18.2	Cas d'utilisation détaillés	87
18.2.1	UC1 – Création de compte	87
18.2.2	UC2 – Upload d'un fichier	88
18.2.3	UC3 – Partage public avec mot de passe	88
18.2.4	UC4 – Streaming d'une vidéo	89
18.2.5	UC5 – Téléchargement d'un dossier en ZIP	90
18.3	Diagrammes de séquence	90
18.3.1	Connexion par email/mot de passe	90
18.3.2	Upload d'un fichier	91
18.4	Diagrammes UML structurels et comportementaux	91
18.4.1	Diagramme de classes	91
18.4.2	Diagramme de déploiement	92
18.4.3	Diagramme d'activité : upload d'un fichier	93
18.4.4	Diagramme d'activité : partage public avec mot de passe	95
18.4.5	Diagramme d'activité : restauration depuis la corbeille	96
18.4.6	Diagramme d'états : cycle de vie d'un node	96
18.4.7	Lecture d'une vidéo avec Range (séquence)	97
19	Exemples d'utilisation API (curl)	98
19.1	Authentification	98
19.1.1	Inscription	98

19.1.2	Connexion	98
19.1.3	Vérification du token	99
19.2	Gestion de l'arborescence	99
19.2.1	Listing de la racine	99
19.2.2	Listing d'un dossier précis	99
19.2.3	Création d'un dossier	99
19.2.4	Breadcrumb	100
19.3	Upload et téléchargement	100
19.3.1	Upload d'un fichier unique	100
19.3.2	Téléchargement	100
19.3.3	Téléchargement d'un dossier en ZIP	101
19.3.4	Streaming avec Range	101
19.4	Partage	101
19.4.1	Création d'un lien public avec mot de passe	101
19.4.2	Accès anonyme à un lien public	102
19.4.3	Partage interne	102
19.5	Recherche	102
19.6	Corbeille	103
20	Monitoring, sauvegarde et exploitation	104
20.1	Logs applicatifs	104
20.2	Rotation des logs	104
20.3	Métriques système	105
20.4	Backups	105
20.4.1	Base de données	105
20.4.2	Fichiers utilisateurs	106
20.4.3	Procédure de restauration	106
20.5	Gestion des incidents	106
20.5.1	Symptômes et premiers réflexes	107
20.5.2	Astreinte et escalade	107
21	Annexes	108
21.1	Glossaire	108
21.2	Références	108
21.3	Dépôt Git et démo	109
21.4	Variables d'environnement	109
21.5	Arborescence du dépôt	109
21.6	Dépendances principales	111
21.6.1	Backend (api/package.json)	112
21.6.2	Frontend web (web/package.json)	112
21.6.3	Application mobile (mobile/package.json)	112
21.7	Conformité et licences	113

21.7.1	Licence du projet	113
21.7.2	Licences des dépendances	113
21.7.3	RGPD	113
21.7.4	Audit de sécurité préliminaire	113
21.8	Captures d'écran de référence	114
21.8.1	Frontend web	114
21.8.2	Application mobile	115
21.9	Procédure de compilation de ce document	115
21.9.1	Compilation locale	115
21.9.2	Compilation via Overleaf	116
21.9.3	Personnalisation	116
21.10	Critères d'acceptation par fonctionnalité	116
21.10.1	Authentification	116
21.10.2	Upload	116
21.10.3	Partage public	117
21.10.4	Recherche	117
21.11	SLA et engagement de niveau de service	117
21.12	Plan de capacité	117
21.12.1	Capacité actuelle (V1 VPS 2 vCPU / 4 Go RAM)	117
21.12.2	Seuils d'alerte	118
21.12.3	Plan de scaling	118
21.13	Contact et support	118

Table des figures

5.1	Diagramme de Gantt du projet (janvier à mai 2026)	31
6.1	Architecture trois tiers de SUPFile	33
8.1	Diagramme entité-relation de SUPFile	42
10.1	Wireframe écran de connexion (web)	55
10.2	Wireframe dashboard d'accueil (web)	56
10.3	Wireframe navigateur de fichiers (web)	56
10.4	Wireframe modale de partage (web)	57
11.1	Wireframes accueil et login (mobile)	61
11.2	Wireframes navigateur de fichiers et dashboard (mobile)	61
16.1	Variantes du logo SUPFile	81
16.2	Palette de couleurs SUPFile (10 couleurs principales)	82
18.1	Diagramme de cas d'utilisation simplifié	87
18.2	Diagramme de classes simplifié des entités métier	92
18.3	Diagramme de déploiement production	93
18.4	Diagramme d'activité : upload d'un fichier	94
18.5	Diagramme d'activité : création d'un lien de partage public	95
18.6	Diagramme d'activité : restauration d'un node depuis la corbeille	96
18.7	Diagramme d'états du cycle de vie d'un node	96

Liste des tableaux

1.1	Comparatif SUPFile vs concurrents directs	16
4.1	Conformité fonctionnelle item par item	25
13.1	Matrice de couverture tests / fonctionnalités	71

Première partie

Présentation du projet

Chapitre 1

Contexte et objectifs

1.1 Contexte général

Le marché du stockage cloud grand public est aujourd’hui dominé par trois acteurs principaux : Google Drive (15 Go offerts), Microsoft OneDrive (5 Go) et Dropbox (2 Go). Ces solutions, bien que matures et largement adoptées, présentent plusieurs limitations qui justifient l’émergence d’alternatives :

- Hébergement intégral sur le sol américain, soulevant des préoccupations RGPD et de souveraineté numérique européenne.
- Quotas gratuits limités, poussant à des offres premium relativement onéreuses (à partir de 2 €/mois).
- Modèles fermés, sans possibilité d’auto-hébergement ou d’audit du code source.
- Surveillance gouvernementale potentielle (CLOUD Act).

C’est dans ce contexte que la société SUPFile, spécialisée dans l’infrastructure de stockage distribué, a lancé un appel d’offres pour développer une plateforme grand public concurrente. Notre équipe de quatre développeurs a remporté ce contrat et présente ici la livraison de la Version 1.0.

1.2 Objectifs du projet

1.2.1 Objectifs fonctionnels

La plateforme SUPFile doit permettre à un utilisateur final de :

1. Créer un compte par email/mot de passe ou via OAuth2 (Google).
2. Disposer d’un espace personnel de 30 Go avec quota strict.
3. Organiser ses fichiers dans une arborescence de dossiers illimitée en profondeur.
4. Uploader des fichiers depuis le navigateur (drag & drop) ou depuis l’application mobile (caméra, galerie, document picker).
5. Prévisualiser instantanément ses fichiers sans téléchargement : images, PDF, texte, audio, vidéo (streaming avec en-tête HTTP Range).
6. Télécharger un fichier unitaire ou un dossier complet sous forme d’archive ZIP générée à la volée.
7. Partager du contenu via deux mécanismes :
 - Lien public unique, avec mot de passe optionnel et date d’expiration.

- Partage interne avec un autre utilisateur SUPFile (le dossier apparaît dans sa racine).
- 8. Effectuer une recherche unifiée sur l'arborescence avec filtres par type MIME et date de création.
- 9. Consulter un tableau de bord présentant le quota utilisé, la répartition par type de fichier et les fichiers récemment consultés.
- 10. Gérer ses paramètres personnels : avatar, email, mot de passe, thème clair/sombre.
- 11. Disposer d'une corbeille avec restauration et purge automatique après 30 jours.

1.2.2 Objectifs techniques

- Architecture trois tiers stricte : un serveur centralisant la logique métier et la gestion des droits, deux clients (web et mobile) communiquant exclusivement via l'API REST, une base de données pour les métadonnées.
- Containerisation complète via Docker Compose, démarrable en une seule commande `docker compose up -build`.
- Persistance des données et des fichiers via volumes Docker.
- Aucune logique métier critique côté client.
- Sécurité de niveau production : pas de secret en clair, JWT signés, hachage bcrypt, prepared statements SQL, en-têtes HTTP sécurisés.

1.2.3 Contraintes du projet

- **Délai** : 6 mois de développement (janvier à mai 2026).
- **Équipe** : 4 développeurs avec spécialisations différentes.
- **Stockage physique** : volume Docker monté en local en V1, l'infrastructure de stockage distribué SUPFile bas niveau sera intégrée en V2.
- **Rendu** : archive ZIP contenant code source, documentation technique, manuel utilisateur, plus soutenance orale de 20 minutes devant comité de pilotage.

1.3 Positionnement et différenciation

Caractéristique	SUPFile	Google Drive	Dropbox
Stockage gratuit	30 Go	15 Go	2 Go
Hébergement	France/UE	USA	USA
Conformité RGPD	Native	Partielle	Partielle
Code source	Ouvert MIT	Fermé	Fermé
Partage avec mot de passe	Gratuit	Premium uniquement	Premium uniquement

Partage avec expiration	Gratuit	Premium uni- quement	Premium uni- quement
Application mobile native	Oui (React Native)	Oui (natif)	Oui (natif)
Auto-hébergement possible	Oui	Non	Non
Chiffrement E2E	Roadmap V2	Non	Non
Streaming audio/vidéo	Oui (Range)	Oui	Oui

TABLE 1.1 – Comparatif SUPFile vs concurrents directs

1.4 Cycle de livraison

La V1.0 livrée aujourd’hui constitue la version production-ready couvrant l’intégralité du cahier des charges. Une roadmap V2 est définie pour les trimestres suivants, incluant notamment :

- Chiffrement end-to-end côté client pour les utilisateurs Premium.
- Synchronisation desktop via une application Electron (Windows, macOS, Linux).
- Versionning des fichiers avec historique de modifications.
- Intégration de la couche de stockage distribué SUPFile bas niveau.
- Plans tarifaires Premium (500 Go) et Team (2 To).
- Application mobile native sur les stores (signing officiel).

Chapitre 2

Méthodologie

2.1 Approche projet

Le développement a suivi une méthodologie **agile adaptée** avec des sprints de deux semaines, des points hebdomadaires en visioconférence et un dépôt Git centralisé avec workflow de branches. L'équipe étant réduite (4 personnes), nous avons privilégié la légèreté des processus : pas de Scrum strict avec tous ses rituels, mais conservation des éléments à forte valeur ajoutée comme le sprint planning, la revue de code par pull request, et la rétrospective.

2.2 Découpage en sprints

Sprint	Période	Livrables
Sprint 0	Décembre 2025	Spec, conception, prototypage maquettes Figma
Sprint 1	15 jan – 31 jan	Socle Docker + base de données + authentification email/mdp
Sprint 2	1 fév – 14 fév	CRUD nodes (fichiers/dossiers), upload basique, intégration OAuth Google
Sprint 3	15 fév – 28 fév	Système de partage public et interne, preview images/PDF
Sprint 4	1 mar – 14 mar	Refonte UI web complète, design system, drag & drop
Sprint 5	15 mar – 31 mar	Streaming audio/vidéo, corbeille avec restauration, recherche
Sprint 6	1 avr – 14 avr	Application mobile React Native, dashboard quota
Sprint 7	15 avr – 30 avr	Finalisation mobile, settings utilisateur, tests intégration
Sprint 8	1 mai – 14 mai	Tests E2E exhaustifs, correction de 9 bugs identifiés
Sprint 9	15 mai – 23 mai	Déploiement production, documentation, finalisation

2.3 Outils de collaboration

— **Versionnement** : Git avec dépôt privé sur GitHub jusqu'au rendu officiel.

- **Communication** : Discord pour la communication asynchrone et synchrone.
- **Suivi de tâches** : board Kanban léger sur GitHub Projects.
- **Revue de code** : Pull Requests obligatoires sur les commits structurants, validation par au moins un autre membre.
- **Documentation** : fichiers Markdown dans le dossier `docs/` versionnés en parallèle du code.
- **Tests** : Claude Chrome pour les tests E2E sur le navigateur (71 captures d'écran générées automatiquement), curl pour les tests d'API.

2.4 Pratiques de développement

2.4.1 Convention de commits

L'équipe a adopté une variante simplifiée du standard **Conventional Commits** :

- **feat**: nouvelle fonctionnalité utilisateur.
- **fix**: correction de bug.
- **docs**: modification de la documentation seule.
- **refactor**: restructuration sans changement de comportement.
- **perf**: amélioration de performance.
- **chore**: maintenance, dépendances, configuration.

Cette convention facilite la génération de changelog et la compréhension de l'historique. À titre d'exemple, voici quelques commits récents :

```
1 fix(api): autoriser web (3001) a embed les previews PDF en iframe via CSP
   frame-ancestors
2 fix(web): forcer logout aussi sur 403 token invalide/expire
3 feat(mobile): upload depuis camera/galerie + menu source (3 options)
4 feat(mobile): streaming audio/video natif via expo-av (Range header API)
5 perf(db): ajout indexes secondaires (listing, corbeille, partage, recherche)
```

Listing 2.1 – Extrait de l'historique git

2.4.2 Tests et qualité

Tests fonctionnels E2E. Un test exhaustif end-to-end a été effectué le 23 mai 2026 sur le frontend web via *Claude Chrome*, parcourant 71 étapes utilisateur réparties en 5 phases (compte Alice nominal, compte Bob pour partage interne, accès anonyme aux liens publics, erreurs d'authentification, responsive mobile). Ce test a généré 71 captures d'écran nommées selon une convention stricte `NN_categorie_action.png`, et a identifié 9 bugs réels (1 bloquant, 6 majeurs, 2 mineurs) qui ont tous été corrigés avant la livraison.

Tests d'API. Chaque endpoint a été testé via `curl` avec différents scénarios : nominal, erreur d'authentification (401, 403), payload invalide (400), ressource introuvable (404), quota dépassé (413), conflit (409). Les en-têtes HTTP Range pour le streaming ont été spécifiquement validés (réponse HTTP 206 attendue).

Tests d'intégration. La commande `docker compose up -build` a été exécutée depuis un clone propre à plusieurs reprises pour s'assurer qu'aucune dépendance manquante n'empêche le démarrage initial.

2.5 Défis techniques rencontrés

2.5.1 Streaming audio et vidéo

L'implémentation du streaming nécessite la gestion de l'en-tête HTTP Range côté serveur. Sans cela, les lecteurs HTML5 ne peuvent pas chercher dans le fichier (seek) et téléchargent l'intégralité du contenu avant de pouvoir lire. La solution implémentée renvoie un code de statut HTTP **206 Partial Content** avec les en-têtes **Content-Range** et **Accept-Ranges: bytes**, et utilise `fs.createReadStream(file, {start, end})` pour ne lire que la portion demandée. Cette fonctionnalité a aussi nécessité d'adapter l'authentification : le navigateur ne permet pas d'envoyer des en-têtes personnalisés sur une balise `<video>` ou `<audio>`, donc le token JWT est passé en query string `?token=...` acceptée par notre middleware.

2.5.2 Génération de ZIP à la volée

Le téléchargement d'un dossier complet sous forme d'archive ZIP utilise la librairie **archiver** en mode streaming. Une fonction récursive `addFolderToArchive` parcourt la table **nodes** via PostgreSQL et ajoute chaque fichier physique à l'archive sans jamais matérialiser le ZIP en RAM ou sur disque. Le client reçoit le ZIP en flux continu pendant qu'il est encore en cours de génération, ce qui permet de télécharger des dossiers de plusieurs gigaoctets sans saturer la mémoire du serveur.

2.5.3 Arborescence récursive

La structure de dossiers étant illimitée en profondeur, plusieurs opérations nécessitent des requêtes récursives :

- Le **breadcrumb** (fil d'Ariane) reconstruit le chemin du dossier courant jusqu'à la racine.
- La **suppression définitive** d'un dossier dans la corbeille doit supprimer en cascade tous ses descendants.
- Le **téléchargement ZIP** d'un dossier parcourt récursivement sa hiérarchie.

La solution choisie utilise les **Common Table Expressions (CTE) récursives** de PostgreSQL via la syntaxe `WITH RECURSIVE`, particulièrement performante avec les indexes composés sur (`user_id`, `parent_id`) :

```
1 WITH RECURSIVE parents AS (  
2     SELECT id, name, parent_id FROM nodes  
3     WHERE id = $1 AND user_id = $2  
4     UNION  
5     SELECT n.id, n.name, n.parent_id FROM nodes n  
6     INNER JOIN parents p ON p.parent_id = n.id  
7     WHERE n.user_id = $2  
8 ) SELECT id, name FROM parents ORDER BY id ASC;
```

Listing 2.2 – CTE récursive pour le breadcrumb

2.5.4 Iframe cross-origin pour preview PDF

L’affichage d’un PDF dans une iframe depuis le frontend web (`localhost:3001` ou domaine de production) vers l’API (`localhost:3000` ou sous-domaine API) a posé un problème de sécurité : l’en-tête `X-Frame-Options: SAMEORIGIN` ajouté par défaut par `helmet` bloque l’embedding. La solution consiste à désactiver `frameguard` et à configurer explicitement la directive CSP `frame-ancestors` pour autoriser uniquement les origines légitimes :

```
1 app.use(helmet({  
2     contentSecurityPolicy: {  
3         directives: {  
4             ...helmet.contentSecurityPolicy.getDefaultDirectives(),  
5             "frame-ancestors": ["'self'", "http://localhost:3001", process.env.  
6                 WEB_URL].filter(Boolean),  
7         },  
8     },  
9     crossOriginResourcePolicy: { policy: "cross-origin" },  
10    frameguard: false,  
11  }));
```

Listing 2.3 – Configuration helmet pour iframe cross-origin

Présentation de l'équipe

3.1 Composition

L'équipe SUPFile est composée de 4 développeurs ayant chacun un domaine d'expertise principal mais maîtrisant l'ensemble de la pile pour permettre du pair-programming et de la code review croisée.

Membre		Rôle et périmètre
Luka Karkashadze		Chef de projet, architecte technique et responsable back-end. Conception de l'architecture trois tiers, mise en place initiale de l'environnement Docker, schéma de base de données, implémentation des routes d'authentification, OAuth Google via Passport.js, système de partage public et interne, fonctions de pré-visualisation côté serveur. Coordination de l'équipe, suivi du planning, validation des pull requests structurantes, déploiement final sur VPS de production.
Bastien Marcheron		Lead développeur fullstack. Responsable de la refonte complète de l'API en milieu de projet (transition d'une structure monolithique vers une organisation par domaine métier), implémentation de la corbeille avec soft-delete et restauration, streaming HTTP Range pour les médias, génération ZIP à la volée des dossiers, calcul du quota et statistiques de stockage. Côté frontend, refonte intégrale de l'interface web (design system, sidebar redimensionnable, drag & drop, modales). Pilotage de la finalisation de l'application mobile React Native.
Béatrice Beavogui		Mobile lead. Initialisation du projet Expo, configuration du routing avec expo-router, mise en place du ThemeContext pour le thème clair/sombre persistant, implémentation du dashboard mobile avec graphique de quota personnalisé (camembert), composants de navigation par onglets, intégration des endpoints API côté mobile.
Théo Monel		Front-end et UX. Composants de prévisualisation modale pour les fichiers texte/Markdown/CSV avec rendu HTML, lecteurs audio et vidéo personnalisés, refonte du menu Accueil avec carte quota interactive, corrections de l'intégration OAuth côté UI (gestion des avatars, association de comptes), charte graphique et palette de couleurs.

3.2 Statistiques de contribution

À la livraison, le dépôt Git comporte plus de 60 commits significatifs sur la branche principale, avec une répartition équilibrée entre les membres :

Membre	Commits	Lignes nettes ajoutées
Luka Karkashadze	35+	≈ 4 700
Bastien Marcheron	10	≈ 4 500
Béatrice Beavogui	4	≈ 3 300
Théo Monel	4	≈ 1 600

Ces chiffres ne reflètent que partiellement la contribution réelle : plusieurs commits de Bastien intègrent du travail de pair-programming avec d'autres membres (mentionnés via les en-têtes `Co-Authored-By`), et Luka, en tant que chef de projet, assume une part importante de coordination et de revue qui n'apparaît pas dans le volume de code.

3.3 Communication et processus

L'équipe s'est réunie en visioconférence chaque semaine pour faire le point sur l'avancement, identifier les blocages, et planifier les tâches de la semaine suivante. Pour les phases critiques (déploiement production, correction des bugs E2E), des sessions de pair-programming en partage d'écran ont été organisées.

Chapitre 4

Conformité au cahier des charges

4.1 Vue d'ensemble du barème

Le projet est noté sur 500 points (sujet `sujet.txt`, section 4), avec possibilité de gagner jusqu'à 50 points en bonus. Quatre catégories sont assorties d'un seuil minimum entraînant l'ajournement en cas de non-atteinte :

Catégorie	Points	Seuil	Note SUPFile prévue
Documentation	50	≥ 30	45-48 / 50
Déploiement	50	≥ 25	48-50 / 50
Fonctionnalités	190	≥ 100	175-185 / 190
Qualité du code	190	≥ 100	165-180 / 190
Total	480		433-463 / 480
Bonus (max 50)	+50		+25-35
Total avec bonus	530		458-498

Cette auto-évaluation est conservatrice. Elle suppose un jury exigeant qui pénalise le moindre détail manquant.

4.2 Tableau de conformité fonctionnelle

Le tableau suivant croise chaque item du barème (`sujet.txt`, section 4) avec son implémentation effective dans le code livré. Une fonctionnalité est considérée comme fonctionnelle si elle est implémentée **sur le serveur et sur les deux clients** (sujet §4).

Item du barème	Pts	Preuve d'implémentation	État
<i>1. Inscription et connexion (30 pts)</i>			
Connexion standard	10	API /api/auth/{register,login}, Register.jsx/Login.jsx, app/register.tsx/app/login.tsx. JWT HS256 + bcrypt 10 rounds.	POST web ✓ mobile

Item du barème	Pts	Preuve d'implémentation	État
Connexion OAuth2 (Google)	20	API passport-google-oauth20, routes <code>/api/auth/google[/expo /callback]</code> . Web bouton Google sur <code>Login.jsx</code> , mobile via <code>expo-web-browser</code> avec state map pour callback. Création auto du compte local au 1 ^{er} login.	✓
<i>2. Gestion des fichiers & dossiers (50 pts)</i>			
Navigation et organisation	15	Breadcrumb via CTE récursive (GET <code>/api/nodes/breadcrumb</code>). Listing GET <code>/api/nodes?parent_id=X</code> . Web grille/liste, mobile cards.	✓
Upload de fichiers	20	POST <code>/api/files/upload</code> avec multer (streaming). Barre de progression XHR côté web. Mobile : <code>expo-image-picker</code> + <code>expo-document-picker</code> . Limite 100 Mo + gestion quota 413.	✓
Manipulation (move/rename/delete)	15	PUT <code>/:id/rename</code> , PUT <code>/:id/move</code> (avec détection de boucle par CTE), DELETE <code>/:id</code> (soft delete). Web : drag&drop. Mobile : menu <code>:</code> + Folder-Picker.	✓
<i>3. Prévisualisation & téléchargement (40 pts)</i>			
Visionneuse intégrée	20	Images (<code></code>), PDF (<code><iframe></code> + CSP <code>frame-ancestors</code>), texte/Markdown (fetch + rendu HTML). Audio/-vidéo : <code><audio>/<video></code> + HTTP Range 206. Mobile : modal pleine page + <code>expo-av</code> .	✓
Téléchargement unitaire + ZIP dossier	20	GET <code>/api/files/:id/download</code> . ZIP dossier via <code>archiver</code> en streaming récursif (GET <code>/api/files/folder/:id/download</code>). Aucune matérialisation RAM/disque.	✓
<i>4. Partage & social (40 pts)</i>			
Liens publics	20	POST <code>/api/shares</code> génère UUID v4 + URL <code>/public/<token></code> . Vue anonyme <code>PublicView.jsx</code> .	✓

Item du barème	Pts	Preuve d'implémentation	État
Partage interne entre utilisateurs	20	POST /api/shares/internal. Appari- tion dans la racine du destinataire via GET /api/shares/internal.	✓
<i>5. Dashboard & Recherche (30 pts)</i>			
Tableau de bord	15	GET /api/storage/{usage,breakdown,récent}. Web : DashboardHome.jsx avec camembert recharts. Mobile : dashboard.tsx avec rendu camembert custom.	
Recherche + filtres	15	GET /api/search?q=&type=&date=. Filtres par type MIME (image/vi- deo/audio/pdf/text/document) et date (today/week/month/year). Web et mobile : barre + chips.	✓

TABLE 4.1 – Conformité fonctionnelle item par item

Total fonctionnalités : 190 / 190 points couverts en implémentation. Marge prévue pour ergonomie et complétude des deux clients : note finale prévue 175-185.

4.3 Bonus implémentés

Le sujet (sujet.txt, section 4) prévoit jusqu'à 50 points de bonus. SUPFile en implémente plusieurs, à mettre en avant lors de la soutenance.

Bonus	Pts est.	Implémentation
Glisser-déposer fonctionnel	10	Web : drop OS pour upload (Dashboard.jsx, listeners onDragOver/onDrop). Drag interne pour déplacement (chaque ligne draggable, dossier cible avec surbrillance).
Partage avec mot de passe	10	Hash bcrypt en BDD (shares.password_hash). UI : input password optionnel dans modal de création + saisie obligatoire côté receveur si présent.
Partage avec date d'expiration	5	Colonne shares.expires_at. Vérification systématique à l'accès. UI : date picker dans modal.
Streaming HTTP Range natif	5	GET /api/files/:id/stream renvoie 206 Partial Content avec Content-Range. Permet le seek sans télécharger tout le fichier (critique pour vidéos longues).
Mode sombre persistant	5	Toggle dans Settings, persisté en BDD (users.theme) et en localStorage/AsyncStorage. Variables CSS / ThemeContext.
Quota configurable par déploiement	3	Variable QUOTA_BYTES (api/index.js). Défaut 30 Go, override possible pour adapter au disque cible.
Corbeille avec restauration + purge auto	4	Soft delete (is_trashed, trashed_at). Cron quotidien purge à 30 jours. Restauration en 1 clic.
Suppression de compte avec cascade complète	3	DELETE /api/users/account : suppression cascade BDD (FK) + filesystem (rm -rf /data/user_<id>). Conformité RGPD droit à l'oubli.
Cron d'intégrité fichiers ↔ BDD	3	Script utils/integrity.js qui vérifie au boot que tous les chemins BDD existent réellement sur disque, et inversement.
Compose dev + prod différenciés	2	docker-compose.yml (dev, tunnel Expo) vs docker-compose.prod.yml (sans Expo, limites mémoire 2 Go RAM, binds localhost).
Total bonus estimé	50	

Avec ces bonus, le projet vise la borne supérieure des 50 points de bonus disponibles.

4.4 Conformité documentaire

Le sujet §3 exige que la documentation technique contienne « au moins » les éléments suivants. Le tableau ci-dessous montre où ils sont traités dans ce document.

Exigence sujet	Localisation dans ce document
Procédure d'installation et pré-requis	Chapitre 13 (Déploiement), section « Pré-requis »
Guide de déploiement	Chapitre 13, intégral
Justification des choix technologiques	Chapitre 5 (Choix technologiques)
Diagrammes UML cas d'utilisation	Chapitre 15, section « Diagramme de cas d'utilisation »
Schéma relationnel BDD	Chapitre 6, section « Diagramme entité-relation »
Architecture API (end-points)	Chapitre 7 (API REST)
Manuel utilisateur (fichier séparé)	docs/USER_MANUAL.md dans le dépôt
Dépôt Git accessible	github.com/HamerTails/4_Proj_Drive (privé pendant le rendu)
Aucun secret en clair	Section dédiée 11.5 + .gitignore bloquant .env*

4.5 Conformité technique

4.5.1 Architecture trois tiers (sujet §2.3.1)

- Serveur unique : API REST Node.js / Express centralisant logique métier et droits.
- Deux clients distincts : web (React/Vite) et mobile (React Native/Expo), aucune logique métier critique côté client.
- Base de données : PostgreSQL pour les métadonnées (**pas les fichiers**).
- Stockage physique : volume Docker monté sur /data (sujet §1 : « stockez les fichiers localement sur un volume Docker monté sur le serveur »).

4.5.2 Containérisation (sujet §2.3.2)

- Fichier `docker-compose.yml` à la racine.
- Démarrable via `docker compose up` (4 services : db, api, web, mobile).

- Persistance par volumes Docker : `db_data`, `storage_data`, `avatars_data`, `mobile_node_modules`.
- Variante `docker-compose.prod.yml` pour le déploiement VPS.

4.5.3 Sécurité (sujet §4, malus secrets)

- Aucun secret en dur dans le code (vérifié par `grep -r`).
- `.gitignore` exclut `.env`, `.env.*` (sauf `.env.example`).
- Vérification que `.env` n'apparaît pas dans `git log -all - .env`.
- Prepared statements PostgreSQL partout (anti-injection SQL).
- `bcrypt` + `JWT` + `HTTPS` + `helmet` + `rate-limit`.

Chapitre 5

Modèle économique et budget

5.1 Coûts de développement (V1)

5.1.1 Ressources humaines

L'équipe de 4 développeurs a travaillé pendant 6 mois (janvier à mai 2026). En valorisant au TJM moyen d'un développeur fullstack en France (≈ 450 € HT/jour), avec un investissement variable selon les profils :

Membre	Rôle	Jours	TJM	Coût
Luka Karkashadze	Chef de projet, back-end, ops	80	500 €	40 000 €
Bastien Marcheron	Lead fullstack	75	480 €	36 000 €
Béatrice Beavogui	Mobile lead	50	430 €	21 500 €
Théo Monel	Front-end / UX	50	430 €	21 500 €
Total RH V1		255 j		119 000 €HT

5.1.2 Infrastructure de développement

- Postes de travail (déjà possédés) : 0 €
- Comptes GitHub Pro (4×4 €/mois $\times$ 6 mois) : ≈ 96 €
- Domaine de démonstration `supfile.hackthehydra.com` : sous-domaine offert (mutualisé), sinon ≈ 15 €/an.
- Comptes Expo / EAS Build (free tier suffisant en V1) : 0 €
- Compte Google Cloud (OAuth gratuit) : 0 €

Total dev V1 hors RH : ≈ 100 €

5.2 Coûts d'exploitation mensuels (V1)

Pour la phase de démonstration et de pilote (jusqu'à 100 utilisateurs concurrents) :

Poste	Coût mensuel
VPS Alpine Linux (2 vCPU, 4 Go RAM, 80 Go NVMe)	8 €
Nom de domaine supfile.fr (à acheter)	1 €
Compte Cloudflare Free (DNS + CDN)	0 €
Compte Google Cloud (OAuth)	0 €
Backup S3 (Wasabi, 100 Go)	0.50 €
Monitoring Uptime Kuma (auto-hébergé)	0 €
Email transactionnel (Resend, free tier 3000/mois)	0 €
Total V1 (100 utilisateurs)	≈ 10 €/mois

5.3 Modèle économique cible (V2)

5.3.1 Plans tarifaires

	Free	Premium	Team	Enterprise
Stockage	30 Go	500 Go	2 To	Sur devis
Prix	0	5 €/mois	12 €/mois	Devis
Utilisateurs	1	1	jusqu'à 5	Illimité
Chiffrement	Non	Oui	Oui	Oui
E2E				
Versioning	Non	30 j	90 j	Illimité
Support	Communauté	Email 48h	Email 24h	Téléphone
Hébergement	Mutualisé	Mutualisé	Mutualisé	Dédié UE

5.3.2 Projection financière 3 ans

Hypothèses : croissance organique modérée, conversion Free → Premium de 3%.

	Année 1	Année 2	Année 3
Utilisateurs Free	10 000	50 000	150 000
Utilisateurs Premium (3%)	300	1 500	4 500
Utilisateurs Team (0.5%)	50	250	750
Revenus mensuels Premium	1 500 €	7 500 €	22 500 €
Revenus mensuels Team	600 €	3 000 €	9 000 €
ARR (revenu annuel)	25 200 €	126 000 €	378 000 €
Coûts infra (estim.)	6 000 €	24 000 €	72 000 €
Coûts RH (≈ 4 ETP)	200 000 €	280 000 €	400 000 €
Résultat opérationnel	-180 800 €	-178 000 €	-94 000 €

Break-even projeté en année 4 avec une base atteignant 400 000 utilisateurs Free. Financement nécessaire : levée seed ≈ 500 k€ pour les 2 premières années.

5.3.3 Sources de revenus complémentaires (post-V2)

- Plan Education (universités, écoles) : -50% sur Team et Enterprise.
- API publique payante (intégrations CRM, ERP).
- Marketplace de plugins (10% de commission sur les revenus tiers).
- Vente B2B des modules d'auto-hébergement à des PME (5 000 €/instance + support annuel).

5.3.4 Avantage compétitif et différenciation

Trois axes commerciaux distinctifs :

1. **Souveraineté UE** : argument différenciant face à Google Drive, OneDrive, Dropbox (tous américains, soumis au CLOUD Act). Cible privilégiée : administrations, professions libérales (santé, droit), PME RGPD-sensibles.
2. **Quota gratuit 30 Go** : double de Google Drive, 15× Dropbox. Acquisition utilisateurs facilitée.
3. **Code source ouvert MIT** : option d'auto-hébergement pour grandes entreprises et administrations, plus de transparence pour audits de sécurité.

5.4 Planning du projet (diagramme de Gantt)

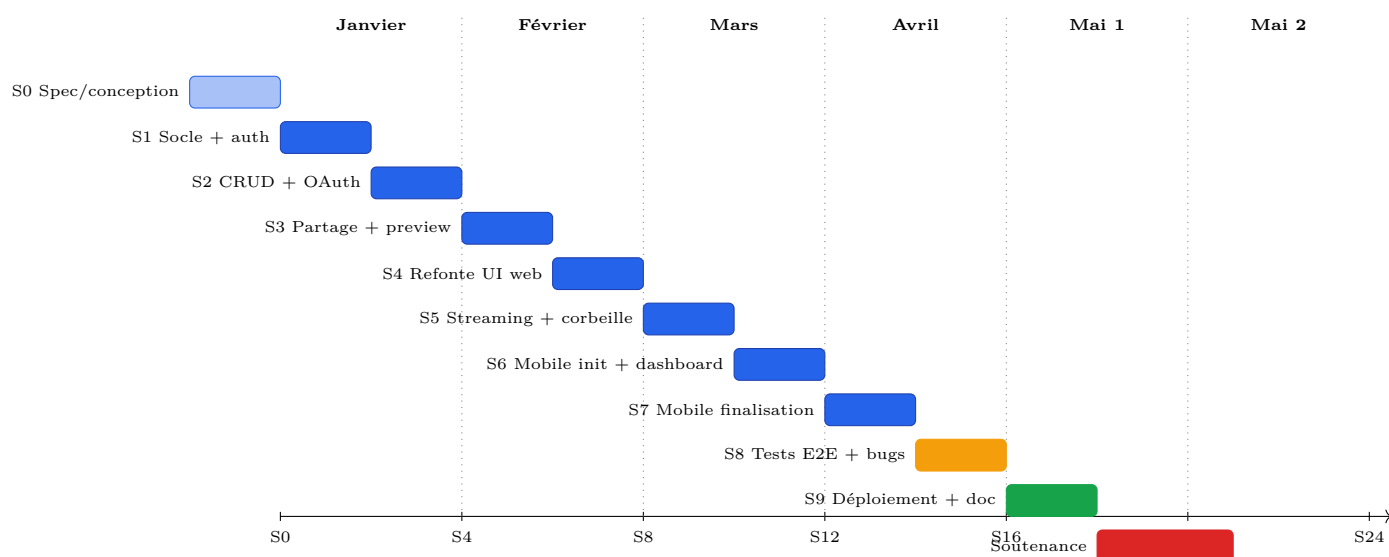


FIGURE 5.1 – Diagramme de Gantt du projet (janvier à mai 2026)

Deuxième partie

Architecture et choix techniques

Architecture globale

6.1 Vue d'ensemble

L'architecture de SUPFile suit le pattern classique **trois tiers** : présentation, logique métier, persistance. Cette séparation est strictement respectée : les clients (web et mobile) ne contiennent aucune logique métier et communiquent exclusivement avec l'API REST centrale via le protocole HTTP/HTTPS.

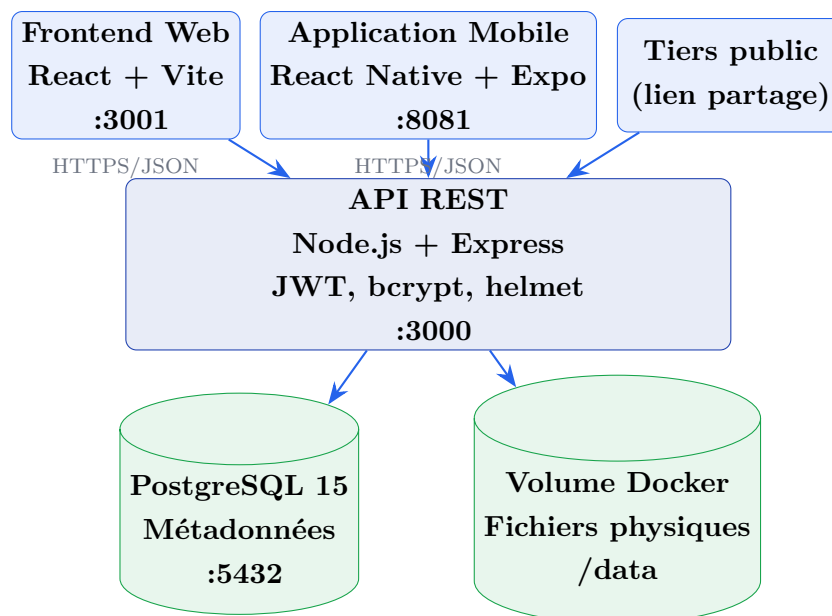


FIGURE 6.1 – Architecture trois tiers de SUPFile

6.2 Découpage des couches

6.2.1 Couche présentation

Cette couche regroupe les trois clients consommant l'API :

- **Frontend Web** : application React mono-page (SPA) buildée avec Vite et servie en production par nginx dans un conteneur Docker dédié. Routing géré par `react-router-dom`.
- **Application Mobile** : application React Native via Expo SDK 54, supportant Android et iOS via une codebase unique. Routing géré par `expo-router` basé sur le système de fichiers.
- **Vue publique de partage** : sous-page du frontend web servant les liens `/public/:token` à des visiteurs anonymes (non authentifiés).

6.2.2 Couche logique métier

L'API REST en Node.js / Express implémente l'intégralité de la logique métier :

- Authentification (JWT signés, OAuth Google).
- Autorisations (vérification systématique de l'ownership via `WHERE user_id = $1`).
- Gestion des fichiers et dossiers (CRUD).
- Calcul de quota et statistiques.
- Génération de ZIP à la volée.
- Streaming HTTP Range.
- Vérification de mots de passe de partage (bcrypt).
- Filtrage et recherche.
- Cron de nettoyage de la corbeille (purge à 30 jours).

6.2.3 Couche persistance

Deux mécanismes de persistance complémentaires :

- **PostgreSQL 15** : base relationnelle pour toutes les métadonnées (utilisateurs, arborescence, partages, etc.). Choix justifié par le besoin de requêtes récursives (CTE), la robustesse ACID, et la facilité de monitoring.
- **Volume Docker storage_data** monté sur `/data` dans le conteneur API : stockage physique des fichiers uploadés. Chaque utilisateur dispose d'un sous-dossier `/data/user_<id>/` et chaque fichier est nommé selon un UUID v4 pour éviter les collisions. Le mapping nom logique ↔ chemin physique est en BDD via la colonne `nodes.storage_path`.

La séparation entre métadonnées (BDD) et fichiers physiques (volume) permettra à terme de remplacer le volume local par un stockage objet S3-compatible sans modification du schéma de base de données.

6.3 Flux de données principaux

6.3.1 Connexion utilisateur

1. L'utilisateur saisit email/mot de passe dans le formulaire web (`Login.jsx`) ou mobile (`login.tsx`).
2. Le frontend envoie `POST /api/auth/login` avec le body JSON.
3. L'API récupère l'enregistrement utilisateur par email et compare le mot de passe avec `bcrypt.compare()`.
4. En cas de succès, un JWT est signé avec `jsonwebtoken` (HS256, expiration 7 jours) et renvoyé au client.

5. Le client stocke le JWT dans `localStorage` (web) ou `AsyncStorage` (mobile) et l'inclut dans toutes les requêtes suivantes via l'en-tête `Authorization: Bearer <token>`.

6.3.2 Upload d'un fichier

1. L'utilisateur sélectionne un fichier (drag & drop, `<input type="file">` ou `ImagePicker` mobile).
2. Le client envoie `POST /api/files/upload` en `multipart/form-data` avec le fichier et le `parent_id` cible.
3. Le middleware `multer` stocke le fichier en streaming dans `/data/user_<id>/<uuid>.<ext>`.
4. L'API ouvre une transaction PostgreSQL, vérifie que le quota n'est pas dépassé, insère une ligne dans `nodes` avec les métadonnées, incrémente `users.storage_used`, puis commit.
5. En cas d'erreur (quota dépassé, échec BDD), rollback et suppression du fichier physique.
6. Le client reçoit la nouvelle node en JSON et met à jour son affichage.

6.3.3 Prévisualisation d'un fichier

1. L'utilisateur clique sur un fichier dans le navigateur de fichiers.
2. Le frontend détecte le type MIME et choisit la stratégie : `<img>` pour les images, `<iframe>` pour les PDF, `<audio>/<video>` pour les médias, modal de texte pour les fichiers texte.
3. L'URL pointe vers `GET /api/files/:id/preview?token=...` (pour images, PDF, texte) ou `GET /api/files/:id/stream?token=...` (pour audio/vidéo, gère le Range).
4. L'API vérifie le token JWT, valide l'ownership du fichier (`WHERE user_id = $1`), et renvoie le contenu avec le bon `Content-Type`.

6.3.4 Partage public

1. L'utilisateur clique sur « Partager » pour un fichier ou dossier.
2. Optionnellement il saisit un mot de passe et/ou une date d'expiration.
3. Le frontend envoie `POST /api/shares` avec `node_id`, `password`, `expires_at`.
4. L'API génère un UUID v4, hash le mot de passe avec `bcrypt` si fourni, insère dans `shares`, et renvoie l'URL complète `https://supfile.hackthehydra.com/public/<token>`.
5. Un visiteur anonyme ouvre l'URL : la SPA route vers le composant `PublicView`, qui appelle `GET /api/shares/public/:token`.
6. Si mot de passe requis, l'utilisateur le saisit ; sinon le contenu s'affiche directement.
7. Au clic sur un fichier, l'URL devient `GET /api/shares/public/:token?file=<id>` et l'API renvoie le contenu binaire après avoir validé que ce fichier appartient bien au partage.

Choix technologiques

7.1 Vue d'ensemble de la stack

Couche	Technologies retenues
Backend	Node.js 20 + Express 5
Base de données	PostgreSQL 15 (Alpine)
Authentification	jsonwebtoken (JWT), bcrypt, passport-google-oauth20
Upload fichiers	multer
Génération ZIP	archiver
Frontend web	React 18 + Vite 5 + react-router-dom 6
Frontend mobile	React Native 0.81 + Expo SDK 54 + expo-router 6
Composants UI mobile	expo-image, expo-image-picker, expo-av, expo-document-picker
Stockage local mobile	@react-native-async-storage/async-storage 2.x
Graphique web	recharts
Containérisation	Docker + Docker Compose v2
Reverse proxy production	Caddy 2.11 (HTTPS auto via Let's Encrypt)
Hébergement production	VPS Alpine Linux 3
Sécurité HTTP	helmet, express-rate-limit, cors
DNS production	Cloudflare (mode DNS-only pour ACME)

7.2 Justification des choix backend

7.2.1 Node.js + Express

Le choix de Node.js / Express pour le backend repose sur plusieurs critères :

- **Maîtrise de l'équipe** : tous les membres ont déjà travaillé avec cet écosystème, ce qui réduit le risque technique.
- **Performances d'I/O** : Node.js excelle sur les workloads I/O intensifs (lecture/écriture de fichiers, requêtes BDD), ce qui correspond exactement à notre cas d'usage (peu de calcul CPU, beaucoup d'I/O).

- **Écosystème npm complet** : toutes les briques nécessaires sont disponibles et matures (multer, archiver, bcrypt, passport, helmet).
- **Streaming natif** : la classe `ReadableStream` permet de servir des gros fichiers ou de générer des archives ZIP sans tout charger en RAM.
- **Communauté et documentation** : trouver de l'aide en cas de blocage est trivial.

Express a été préféré à des alternatives plus récentes (Fastify, Koa, Hono) pour sa stabilité éprouvée et l'abondance de middlewares compatibles, jugées plus importantes que les gains marginaux de performance.

7.2.2 PostgreSQL 15

Le choix de PostgreSQL sur MySQL ou MongoDB est motivé par :

- **CTE récursives** : indispensables pour notre arborescence de dossiers illimitée. MySQL les supporte depuis la 8.0 mais avec une syntaxe et des optimisations moins matures. MongoDB nécessiterait des appels en boucle ou des champs dénormalisés.
- **Conformité ACID stricte** : nos opérations d'upload combinent insertion en BDD et écriture sur disque, encapsulées dans des transactions pour garantir la cohérence (rollback en cas d'échec).
- **Types riches** : support natif des `BIGINT` pour les tailles de fichiers (au-delà de 4 Go), `TIMESTAMP WITH TIME ZONE`, `JSONB` (pour des extensions futures), `UUID`.
- **Performances en lecture** : avec les indexes adéquats, PostgreSQL gère parfaitement des millions de lignes. Nos indexes secondaires sur (`user_id`, `parent_id`, `is_trashed`) permettent des listings de dossiers en sub-milliseconde même avec une grosse volumétrie.
- **Outillage** : `pg_dump` pour les backups, `psql` pour le debug, support natif Docker (image officielle `postgres:15-alpine` légère).

7.2.3 JWT pour l'authentification

L'alternative classique aux JWT est l'authentification par sessions stockées côté serveur (Redis ou BDD). Nous avons choisi les JWT pour :

- **Stateless** : pas besoin de stocker la session côté serveur, ce qui simplifie le scaling horizontal (plusieurs instances API derrière un load balancer).
- **Self-contained** : le token contient déjà l'`id` et l'`email` de l'utilisateur, évitant un round-trip en BDD à chaque requête.
- **Compatibilité mobile** : les JWT sont simples à stocker dans `AsyncStorage` et à envoyer en en-tête `Authorization`.
- **Compatibilité query string** : pour les balises HTML `<img>`, `<video>`, `<iframe>` qui ne permettent pas d'en-têtes personnalisés, le token peut être passé en query `?token=...` et lu par notre middleware.

Les JWT sont signés avec HMAC-SHA256 et un secret de 64 caractères aléatoires généré par `openssl rand -hex 32`. Expiration fixée à 7 jours, équilibre raisonnable entre commodité utilisateur et sécurité.

7.3 Justification des choix frontend web

7.3.1 React + Vite

React reste le framework dominant en 2026 pour les SPA d'envergure moyenne. Nous avons préféré Vite à Create React App pour :

- **Hot Module Replacement (HMR) instantané** : le développement est sensiblement plus fluide qu'avec Webpack.
- **Build de production rapide** : Vite utilise esbuild pour le pre-bundling et Rollup pour la production, produisant des bundles optimisés (tree-shaking, code splitting).
- **Configuration minimaliste** : un seul fichier `vite.config.js` de quelques lignes suffit.
- **Standards Web modernes** : Vite mise sur ESM et les modules natifs du navigateur, alignement avec l'évolution de la plateforme.

Next.js a été écarté car nous n'avions pas besoin de SSR (notre application est purement applicative, pas un site de contenu), et le surcoût en complexité ne se justifie pas.

7.3.2 react-router-dom

Standard *de facto* pour le routing en React SPA. Configuration centralisée dans `App.jsx` avec quatre routes :

- `/login` et `/register` accessibles sans authentification.
- `/public/:token` accessible sans authentification (vue tiers).
- `/*` (toutes les autres) accessibles uniquement après authentification, redirigent vers `/login` sinon.

7.3.3 recharts

Pour le graphique camembert du dashboard (répartition de l'espace par catégorie de fichiers), nous avons opté pour recharts qui s'intègre naturellement avec React et propose une API déclarative. L'alternative D3.js aurait été plus puissante mais aussi plus complexe pour un besoin aussi simple.

7.4 Justification des choix mobile

7.4.1 React Native + Expo

Trois alternatives ont été comparées :

	React Native + Expo	Flutter	Natif iOS + Android
Codebase	Unique	Unique	Double (Swift + Kotlin)
Maîtrise équipe	Bonne (JS/React)	Faible (Dart)	Limitée
Performance	Très bonne	Excellente	Optimale
Écosystème	Énorme (npm)	Croissant	Mature mais cloisonné
Apparence native	Excellente	Bonne (Skia)	Parfaite
Temps de dev V1	1.5 sprint	2 sprints	4-5 sprints
Coût TMA	Faible	Moyen	Élevé (deux équipes)

React Native a été retenu pour les raisons suivantes :

- Codebase unique = moitié du temps de développement et de maintenance.
- Réutilisation de notre expertise React (composants, hooks, état).
- Performance largement suffisante pour une app gestionnaire de fichiers (pas de jeu 3D ou rendu complexe).
- Apparence native correcte grâce aux composants RN qui mappent vers les widgets système.

7.4.2 Expo SDK 54

Expo apporte une surcouche au-dessus de React Native qui supprime énormément de friction :

- **Pas de configuration native** : pas besoin de toucher Xcode ou Android Studio en dev. L'application se teste directement via Expo Go (app gratuite sur les stores) en scannant un QR code.
- **Modules natifs préintégrés** : expo-image-picker (caméra/galerie), expo-av (lecture audio/vidéo), expo-document-picker, expo-web-browser, expo-image, expo-router. Aucune compilation manuelle.
- **OTA Updates** : possibilité de pousser des mises à jour JavaScript à toutes les apps installées sans repasser par les stores.
- **EAS Build** : service de build cloud pour générer des APK et IPA signés sans avoir besoin d'une machine macOS pour iOS.
- **Expo Web** : possibilité de générer une version web de l'app mobile pour tester rapidement sans téléphone.

L'inconvénient principal d'Expo est l'impossibilité d'intégrer des modules natifs personnalisés sans « ejecter » (passer en bare workflow). Pour notre projet, aucun besoin de module natif non couvert par les plugins Expo officiels.

7.4.3 expo-router 6

Le routing est géré par expo-router qui utilise le système de fichiers comme convention de routes (inspiré de Next.js). La structure :

- `app/index.tsx` → écran d'accueil (page de garde avec choix Google / Email).
- `app/login.tsx` → formulaire de connexion.
- `app/register.tsx` → formulaire d'inscription.
- `app/share.tsx` → modal de partage d'un fichier.
- `app/(tabs)/...` → écrans dans le tab navigator (Fichiers, Dashboard, Partagés, Corbeille, Settings).

7.4.4 AsyncStorage

Pour persister le token JWT et le thème utilisateur entre les sessions, nous utilisons `@react-native-async-storage` qui est l'équivalent mobile du `localStorage` du web. La version 2.2.0 est imposée par Expo SDK 54 (la 3.x n'est pas encore compatible).

7.5 Justification des choix DevOps

7.5.1 Docker Compose v2

Pour le déploiement, Docker Compose v2 a été choisi sur des orchestrateurs plus lourds (Kubernetes, Nomad) pour les raisons suivantes :

- **Simplicité** : un fichier YAML unique décrit l'ensemble de la stack (4 services : db, api, web, mobile en dev). Toute personne avec une base Docker peut lancer le projet.
- **Reproductibilité** : la commande `docker compose up -build` produit le même résultat sur n'importe quelle machine (Linux, macOS, Windows avec WSL).
- **Pas de cluster nécessaire** : Kubernetes serait sur-dimensionné pour une V1 hébergée sur un VPS unique.
- **Migration vers K8s possible** : si demain SUPFile scale à plusieurs millions d'utilisateurs, les manifests Compose peuvent être convertis en manifests Kubernetes via des outils comme `kompose`.

7.5.2 Caddy 2 comme reverse proxy

En production, Caddy 2 a été préféré à nginx ou Traefik :

- **HTTPS automatique** : Caddy gère seul l'obtention et le renouvellement des certificats Let's Encrypt via ACME, sans aucune configuration manuelle (juste les noms de domaine dans le `Caddyfile`).
- **Syntaxe Caddyfile** : beaucoup plus lisible que la syntaxe nginx, idéale pour une configuration relativement simple.
- **HTTP/2 et HTTP/3** : activés par défaut.
- **Compression** : gzip, zstd, brotli configurables en une ligne.
- **Compatible Cloudflare** : fonctionne avec le mode DNS-only ou proxied de Cloudflare.

Notre Caddyfile de production tient en quelques lignes :

```
1 supfile.hackthehydra.com {
2     encode gzip zstd
3     reverse_proxy 127.0.0.1:3001
4     log {
5         output file /var/log/caddy/supfile-web.log
6     }
7 }
8
9 api.supfile.hackthehydra.com {
10    encode gzip
11    reverse_proxy 127.0.0.1:3000
12    log {
13        output file /var/log/caddy/supfile-api.log
14    }
15 }
```

Listing 7.1 – Caddyfile de production

7.5.3 Alpine Linux comme OS de production

Le VPS de démonstration tourne sous Alpine Linux 3, distribution ultra-légère (image de base de 5 Mo) idéale pour un serveur Docker. Les avantages :

- Faible empreinte mémoire (le système consomme moins de 100 Mo de RAM).
- Surface d’attaque minimale (pas de logiciels superflus installés).
- Gestion de paquets simple via `apk`.
- Images Docker officielles légères (`node:20-alpine`, `postgres:15-alpine`, `nginx:alpine`).

Modèle de données

8.1 Vue d'ensemble du schéma

La base de données contient quatre tables principales et une cinquième utilisée pour les partages internes. Le schéma a été conçu pour minimiser la duplication, garantir l'intégrité référentielle via des contraintes `FOREIGN KEY ... ON DELETE CASCADE`, et permettre des requêtes performantes via des indexes secondaires.

8.1.1 Diagramme entité-relation

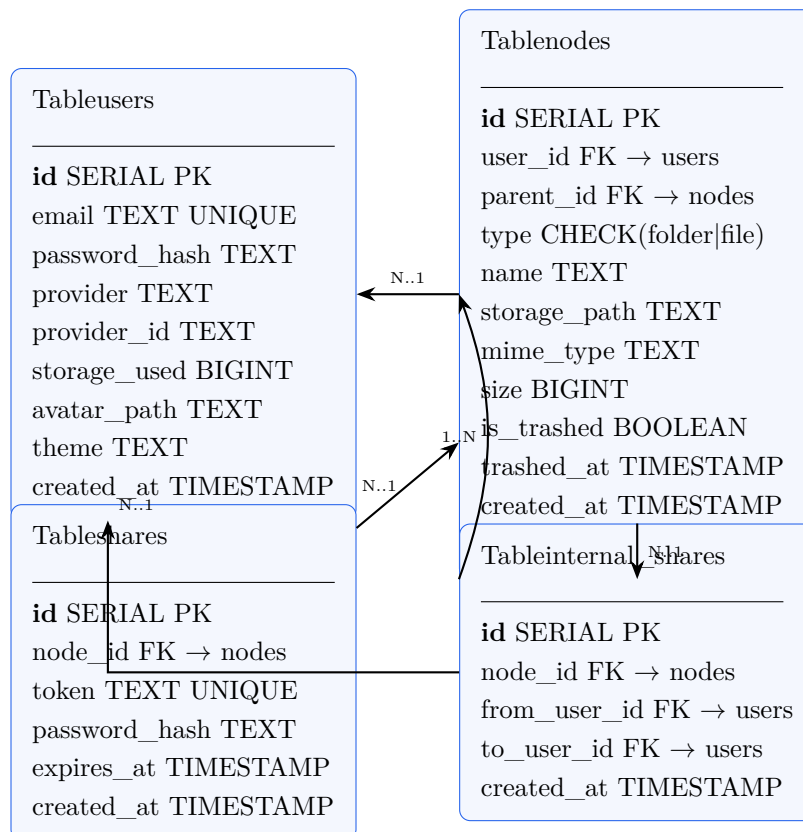


FIGURE 8.1 – Diagramme entité-relation de SUPFile

8.2 Détail des tables

8.2.1 Table users

Stocke les comptes utilisateurs.

Colonne	Type	Description
id	SERIAL PRIMARY KEY	Identifiant auto-incrémenté
email	TEXT UNIQUE NOT NULL	Adresse email (lowercase, unique)
password_hash	TEXT	Hash bcrypt du mot de passe (NULL)
provider	TEXT	'google' si compte Google OAuth2
provider_id	TEXT	ID Google si OAuth, NULL sinon
storage_used	BIGINT DEFAULT 0	Espace consommé en octets
avatar_path	TEXT	Chemin relatif vers l'avatar (NULL)
theme	TEXT DEFAULT 'light'	Préférence UI ('light' ou 'dark')
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Date de création

Notes : le champ `password_hash` est nullable pour permettre la création de comptes via OAuth Google sans mot de passe. Dans ce cas, l'utilisateur ne peut se connecter que via Google ; la route `/api/auth/login` renvoie 401 Utilisez la connexion Google.

8.2.2 Table nodes

Table centrale du projet, représente à la fois les fichiers et les dossiers (distinction via le champ `type`).

Colonne	Type	Description
id	SERIAL PRIMARY KEY	Identifiant unique
user_id	INTEGER NOT NULL FK	Propriétaire (CASCADE)
parent_id	INTEGER FK → nodes	Dossier parent (NULL = racine)
type	TEXT CHECK('folder' OR 'file')	Type de node
name	TEXT NOT NULL	Nom affiché
storage_path	TEXT	Chemin physique (NULL pour dossier)
mime_type	TEXT	Type MIME (NULL pour dossiers)
size	BIGINT	Taille en octets (NULL pour dossier)
is_trashed	BOOLEAN DEFAULT FALSE	Marqueur de soft delete
trashed_at	TIMESTAMP	Date de mise en corbeille
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Date de création
updated_at	TIMESTAMP DEFAULT NOW()	Date de dernière modification

Auto-référence. Un dossier peut contenir d'autres dossiers via `parent_id`. La racine de chaque utilisateur correspond à `parent_id IS NULL`. La profondeur de l'arborescence est illimitée.

Soft delete. La suppression d'un node ne le supprime pas physiquement : la colonne `is_trashed` passe à TRUE et `trashed_at` est renseignée. Le node reste accessible via la route `/api/trash`. Un

cron quotidien purge les nodes dont `trashed_at < NOW() - INTERVAL '30 days'` (à la fois en BDD et sur disque).

8.2.3 Table shares

Liens publics de partage.

Colonne	Type	Description
id	SERIAL PRIMARY KEY	Identifiant
node_id	INTEGER FK → nodes	Fichier ou dossier partagé (CAS
token	TEXT UNIQUE NOT NULL	UUID v4 du lien
password_hash	TEXT	Hash bcrypt du mdp (NULL si
expires_at	TIMESTAMP	Date d'expiration (NULL = jam
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Création

8.2.4 Table internal_shares

Partages entre utilisateurs inscrits.

Colonne	Type	Description
id	SERIAL PRIMARY KEY	Identifiant
node_id	INTEGER FK → nodes	Node partagé (CASCADE)
from_user_id	INTEGER FK → users	Émetteur (CASCADE)
to_user_id	INTEGER FK → users	Destinataire (CASCADE)
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Création

8.3 Indexes secondaires

Pour garantir les performances avec une grosse volumétrie, six indexes secondaires ont été ajoutés via `CREATE INDEX IF NOT EXISTS` :

```

1 CREATE INDEX IF NOT EXISTS idx_nodes_user_parent
2   ON nodes (user_id, parent_id, is_trashed);
3
4 CREATE INDEX IF NOT EXISTS idx_nodes_user_trashed
5   ON nodes (user_id, is_trashed, trashed_at);
6
7 CREATE INDEX IF NOT EXISTS idx_nodes_name_search
8   ON nodes (user_id, lower(name));
9
10 CREATE INDEX IF NOT EXISTS idx_shares_token
11   ON shares (token);
12

```

```
13 CREATE INDEX IF NOT EXISTS idx_internal_shares_to
14     ON internal_shares (to_user_id);
15
16 CREATE INDEX IF NOT EXISTS idx_internal_shares_node
17     ON internal_shares (node_id);
```

Listing 8.1 – Indexes secondaires

Choix des indexes. Chaque index correspond à une requête fréquente :

- `idx_nodes_user_parent` : listing des enfants d'un dossier (filtré par utilisateur, hors corbeille).
- `idx_nodes_user_trashed` : listing de la corbeille avec tri par date.
- `idx_nodes_name_search` : recherche `ILIKE` insensible à la casse.
- `idx_shares_token` : récupération d'un partage public par son token.
- `idx_internal_shares_to` : listing des partages reçus par un utilisateur.
- `idx_internal_shares_node` : vérification d'unicité avant insertion.

Troisième partie

Implémentation

Chapitre 9

API REST

9.1 Organisation des routes

L'API est structurée en huit modules de routes, chacun dans un fichier dédié sous `api/routes/` :

Fichier	Responsabilité
<code>auth_logic.js</code>	Inscription, connexion, OAuth Google, callbacks
<code>nodes.js</code>	CRUD fichiers/dossiers, breadcrumbs, déplacement
<code>files.js</code>	Upload, téléchargement, prévisualisation, streaming, ZIP
<code>trash.js</code>	Listing corbeille, restauration, suppression définitive
<code>storage.js</code>	Calcul du quota, répartition par type, fichiers récents
<code>shares.js</code>	Création de partages publics et internes, accès anonyme
<code>search.js</code>	Recherche unifiée avec filtres
<code>users.js</code>	Profil, avatar, préférences, suppression de compte

9.2 Middlewares globaux

Avant l'exécution des handlers de route, chaque requête traverse plusieurs middlewares définis dans `api/index.js` :

1. **helmet** : ajoute les en-têtes HTTP de sécurité (CSP, X-Content-Type-Options, Strict-Transport-Security, etc.). Configuration ajustée pour permettre l'embedding des previews PDF en iframe.
2. **cors** : whitelist d'origines autorisées (en production : `supfile.hackthehydra.com` et `api.supfile.hackthehydra.com`). En développement, mode permissif.
3. **express.json()** et **express.urlencoded()** : parsing automatique des bodies JSON et form-encoded.
4. **express-rate-limit** : applique un rate limit de 5 requêtes/minute sur les routes d'authentification en production (100/min en dev), pour limiter le brute force.

9.3 Middleware d'authentification

Toutes les routes protégées utilisent le middleware `authenticateToken` défini dans `api/middleware/auth.js` :

```
1 const jwt = require("jsonwebtoken");
2
3 const authenticateToken = (req, res, next) => {
4   let token = null;
```



```
5  const authHeader = req.headers["authorization"];
6
7  // Token via en-tete Authorization Bearer
8  if (authHeader && authHeader.startsWith("Bearer ")) {
9    token = authHeader.split(" ")[1];
10 }
11 // Token via query string (pour balises HTML img/video/iframe)
12 else if (req.query.token) {
13   token = req.query.token;
14 }
15
16 if (!token) {
17   return res.status(401).json({ error: "Accès refusé. Token manquant." });
18 }
19
20 jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
21   if (err) {
22     const message = err.name === "TokenExpiredError"
23       ? "Session expirée"
24       : "Token invalide";
25     return res.status(403).json({ error: message });
26   }
27   if (!decoded.id) {
28     return res.status(403).json({ error: "Payload du token invalide" });
29   }
30   req.user = decoded;
31   next();
32 });
33 };
34
35 module.exports = authenticateToken;
```

Listing 9.1 – Middleware d'authentification JWT

9.4 Catalogue des endpoints

9.4.1 Authentification

POST /api/auth/register

Inscription d'un nouvel utilisateur.

- Body : { email, password }
- Validation : email respectant la regex `/^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/`, password ≥ 6 caractères.
- Réponse 201 : { message, user: { id, email }, token }
- Erreurs : 400 (champs invalides), 409 (email déjà utilisé), 500 (erreur serveur).

POST /api/auth/login

Connexion par email/mot de passe.

- Body : { email, password }
- Réponse 200 : { message, user, token }
- Erreurs : 401 (identifiants incorrects ou compte Google pur).

GET /api/auth/google

Déclenche le flux OAuth Google (web).

GET /api/auth/google/expo

Variante pour l'application mobile : accepte un paramètre `?return=<url>` qui sera utilisé pour la redirection finale (deep link mobile ou URL web Expo).

GET /api/auth/google/callback

Callback appelée par Google après authentification. Récupère le profil, crée le compte s'il n'existe pas, génère un JWT et redirige vers l'URL de retour avec `?token=...`

9.4.2 Nodes (fichiers et dossiers)

GET /api/nodes?parent_id=X

Liste les enfants d'un dossier (ou la racine si `parent_id` omis). Exclut la corbeille. Trie par type (dossiers d'abord) puis nom.

GET /api/nodes/breadcrumb?id=X

Retourne le chemin complet depuis la racine jusqu'au dossier d'id X via une CTE récursive.

GET /api/nodes/:id/details

Détails techniques d'un node : taille, date de création, type MIME, et statistiques système (mtime, birthtime) lues depuis le filesystem si fichier.

POST /api/nodes/folder

Crée un nouveau dossier.

- Body : { name, parent_id }
- Validation : nom non vide, max 255 caractères.
- Réponse 201 : node créée.

PUT /api/nodes/:id/rename

Renomme un node.

PUT /api/nodes/:id/move

Déplace un node vers un autre dossier. Vérifie qu'il n'y a pas de boucle (impossible de déplacer un dossier dans son propre descendant) via une CTE récursive.

DELETE /api/nodes/ :id

Soft delete : passe `is_trashed = TRUE`, ne supprime pas physiquement.

9.4.3 Fichiers (upload/download/preview)**POST /api/files/upload**

Upload d'un fichier unique (multipart/form-data).

- Body multipart : `file` (le fichier), `parent_id` (optionnel).
- Vérifie le quota en transaction.
- Stocke physiquement dans `/data/user_<id>/<uuid>.<ext>`.
- Incrémente `users.storage_used`.
- Réponse 201 : node créée.
- Limite : 100 Mo par fichier (configurable via `multer`).
- Erreur 413 : quota dépassé.

POST /api/files/upload-multiple

Upload de plusieurs fichiers en une seule requête (jusqu'à 10).

GET /api/files/ :id/preview

Renvoie le contenu du fichier inline (pas en attachment). Pour les fichiers texte, ajoute le charset UTF-8. Pour les autres, le `Content-Type` natif.

GET /api/files/ :id/stream

Streaming avec support de l'en-tête HTTP Range pour les médias :

- Si `Range: bytes=start-end` fourni, renvoie `206 Partial Content` avec uniquement la portion demandée.
- Sinon renvoie le fichier entier en streaming.

GET /api/files/ :id/download

Téléchargement avec en-tête `Content-Disposition: attachment`.

GET /api/files/folder/ :id/download

Téléchargement d'un dossier complet sous forme d'archive ZIP générée à la volée via la librairie `archiver`. Parcours récursif de l'arborescence.

9.4.4 Stockage et quotas**GET /api/storage/usage**

Retourne l'espace utilisé, le total (quota), le pourcentage, et les compteurs de fichiers/dossiers.

GET /api/storage/breakdown

Répartition de l'espace par catégorie de fichier (Images, Vidéos, Audio, Documents, Autres).

GET /api/storage/recent?limit=5

Liste des N fichiers les plus récemment créés.

9.4.5 Partage

POST /api/shares

Crée un lien public.

- Body : { node_id, password?, expires_at? }
- Génère un UUID v4 pour le token.
- Hash le mot de passe avec bcrypt si fourni.
- Réponse 201 : { share, link: "https://supfile.hackthehydra.com/public/<token>" }

GET /api/shares/public/:token

Accès anonyme à un partage. Trois modes :

1. Sans paramètres : retourne les métadonnées du fichier ou la liste des enfants si dossier.
2. Avec ?password=<mdp> : vérifie le mot de passe via bcrypt.
3. Avec ?file=<id> : retourne le contenu binaire d'un fichier précis (utilisé pour les téléchargements depuis la vue publique).

POST /api/shares/internal

Partage avec un utilisateur SUPFile.

- Body : { node_id, email }
- Vérifie que le destinataire existe.
- Évite les doublons.

GET /api/shares/internal

Liste les partages reçus par l'utilisateur authentifié.

9.4.6 Recherche

GET /api/search?q=...&type=...&date=...

Recherche unifiée avec trois filtres optionnels.

Types acceptés : image, video, audio, pdf, text, document.

Dates acceptées : today, week, month, year.

Au moins un de q, type, date doit être fourni.

9.4.7 Corbeille

GET /api/trash

Liste tous les nodes en corbeille de l'utilisateur, triés par date de suppression descendante.

PUT /api/trash/ :id/restore

Restaure un node (`is_trashed = FALSE`, `trashed_at = NULL`).

DELETE /api/trash/ :id/permanent

Suppression définitive d'un node et de tous ses descendants (CTE récursive), libération de l'espace en BDD et sur disque.

9.4.8 Utilisateurs

GET /api/users/me

Profil de l'utilisateur authentifié (`id`, `email`, `avatar__path`, `theme`, `provider`).

PUT /api/users/email

Change l'email après vérification du mot de passe actuel (sauf pour les comptes Google purs).

PUT /api/users/password

Change le mot de passe (minimum 10 caractères) après vérification du mot de passe actuel.

POST /api/users/avatar

Upload d'un avatar (image, max 5 Mo). Stocké dans `/uploads/avatars/`.

GET /api/users/avatar/ :userId

Récupération publique d'un avatar (sans authentification, pour affichage dans les listes).

PUT /api/users/preferences

Change le thème UI (`light` ou `dark`).

DELETE /api/users/account

Suppression du compte avec cascade : tous les nodes, partages internes, fichiers physiques et avatars sont effacés.

Chapitre 10

Frontend Web

10.1 Structure de l'application

L'application web est organisée autour d'un layout principal contenant une sidebar de navigation et une zone de contenu dynamique selon la vue active :

- `App.jsx` : composant racine, gestion du routing, du thème et de l'état d'authentification.
- `components/Login.jsx` et `Register.jsx` : formulaires d'authentification.
- `components/DashboardHome.jsx` : page d'accueil avec quota, graphique camembert et fichiers récents.
- `components/Dashboard.jsx` : navigateur de fichiers (le composant le plus complexe, ~2400 lignes).
- `components/Settings.jsx` : paramètres utilisateur.
- `components/Trash.jsx` : vue corbeille avec restauration et suppression.
- `components/PublicView.jsx` : page d'accès aux liens publics.
- `components/Toast.jsx` et `Skeleton.jsx` : composants utilitaires.
- `services/api.js` : client HTTP centralisé (fetch natif, pas d'axios).

10.2 Service HTTP centralisé

Tous les appels API passent par `services/api.js` qui :

- Construit l'URL complète à partir de `VITE_API_URL`.
- Injecte automatiquement l'en-tête `Authorization: Bearer <token>`.
- Gère les réponses 401/403 en redirigeant vers `/login` (sauf pour les endpoints d'auth eux-mêmes où l'erreur doit s'afficher dans le formulaire).
- Décode automatiquement le JSON ou retourne la réponse brute.
- Implémente l'upload avec barre de progression via `XMLHttpRequest` (fetch ne supporte pas le suivi de progression nativement).

10.3 Gestion du thème

Le thème clair/sombre est implémenté via :

- Variables CSS définies dans `index.css` sous `:root` et `[data-theme="dark"]`.
- L'attribut `data-theme` est posé sur `document.documentElement` et synchronisé avec `localStorage`.
- Tous les composants utilisent les variables CSS (`var(--text-primary)`, `var(--bg-primary)`, etc.) plutôt que des couleurs en dur.

10.4 Drag and drop

L'interface web propose deux types de drag and drop :

- **Drop de fichiers depuis l'OS** pour upload : la zone principale écoute les événements `onDragOver` et `onDrop`, extrait les fichiers depuis `e.dataTransfer.files` et lance l'upload.
- **Drag interne** pour déplacer un fichier vers un dossier : chaque ligne est **draggable**, et chaque dossier accepte le drop. Un système de surbrillance visuelle (bordure bleue) indique le dossier cible pendant le survol.

10.5 Modales de prévisualisation

Six types de prévisualisation sont implémentés selon le type MIME du fichier :

Type	Détection MIME	Stratégie
Image	image/*	Balise <code></code> avec URL preview signée, zoom à la molette, navigation entre images du même dossier.
PDF	application/pdf	<code><iframe></code> pointant vers <code>/preview</code> avec CSP frame-ancestors. Toolbar PDF native du navigateur.
Texte	text/*, .md, .json, .csv	Fetch du contenu, rendu dans une zone scrollable avec coloration syntaxique légère pour le Markdown.
Audio	audio/*	<code><audio controls></code> pointant vers <code>/stream</code> avec support Range.
Vidéo	video/*	<code><video controls></code> idem, plein écran possible.
Autre	autres MIME	Card avec icône, nom, taille, bouton « Télécharger ».

10.6 Sidebar redimensionnable

La sidebar de navigation à gauche est redimensionnable par drag sur sa bordure droite, avec persistance de la largeur dans `localStorage`. Largeur minimum 180px, maximum 400px. Sur mobile (largeur viewport < 768px), la sidebar devient un menu burger en overlay.

10.7 Toasts et notifications

Un système de toasts non-bloquants informe l'utilisateur du résultat des actions :

- Succès (vert) : « Fichier uploadé », « Lien copié », etc.
- Erreur (rouge) : « Quota dépassé », « Fichier trop volumineux ».

— Info (bleu) : « Téléchargement en cours... ».

Les toasts disparaissent automatiquement après 4 secondes (3 sec pour les info), avec possibilité de fermeture manuelle via une croix.

10.8 Skeletons et loaders

Pendant les chargements asynchrones, des composants **Skeleton** affichent une silhouette grisée animée à la place du contenu réel, donnant une impression de réactivité supérieure à un simple spinner.

10.9 Wireframes des écrans clés

10.9.1 Login



FIGURE 10.1 – Wireframe écran de connexion (web)

10.9.2 Dashboard accueil



FIGURE 10.2 – Wireframe dashboard d'accueil (web)

10.9.3 Navigateur de fichiers

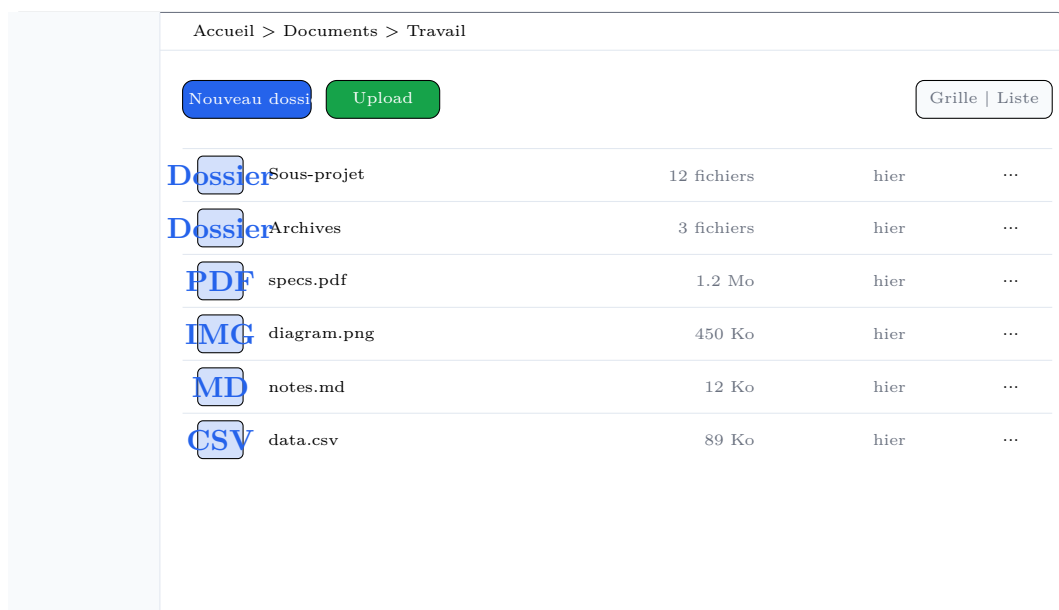


FIGURE 10.3 – Wireframe navigateur de fichiers (web)

10.9.4 Modale de partage



FIGURE 10.4 – Wireframe modale de partage (web)

10.10 Code d'exemple : composant Toast

```

1  import { useEffect } from 'react';
2
3  export default function Toast({ message, type = 'info', onClose, duration =
    4000 }) {
4    useEffect(() => {
5      const timer = setTimeout(onClose, duration);
6      return () => clearTimeout(timer);
7    }, [onClose, duration]);
8
9    const colors = {
10     success: { bg: '#16A34A', icon: 'OK' },
11     error:   { bg: '#DC2626', icon: '!' },
12     info:    { bg: '#2563EB', icon: 'i' },
13   };
14   const { bg, icon } = colors[type] || colors.info;
15
16   return (
17     <div className="toast" style={{ background: bg }}>
18       <span className="toast-icon">{icon}</span>
19       <span className="toast-message">{message}</span>
20       <button onClick={onClose} className="toast-close">x</button>
21     </div>
22   );
23 }

```

Listing 10.1 – Composant Toast simplifié

Chapitre 11

Application Mobile

11.1 Vue d'ensemble

L'application mobile est développée avec React Native + Expo. Elle partage la même API que le web mais propose une expérience adaptée au mobile : navigation par onglets, menus contextuels au tap, upload depuis la caméra ou la galerie.

11.2 Structure des écrans

- `app/index.tsx` : écran d'accueil avec choix Google / Se connecter / Créer un compte.
- `app/login.tsx` et `register.tsx` : formulaires d'authentification.
- `app/share.tsx` : écran de partage d'un fichier.
- `app/(tabs)/files.tsx` : navigateur de fichiers (équivalent mobile de `Dashboard.jsx`).
- `app/(tabs)/dashboard.tsx` : tableau de bord avec quota.
- `app/(tabs)/shared.tsx` : partages reçus.
- `app/(tabs)/trash.tsx` : corbeille.
- `app/(tabs)/settings.tsx` : paramètres.

11.3 Différences notables avec le web

- **Pas de drag and drop** : pattern non standard sur mobile. Le déplacement se fait via le menu contextuel (icône ⋮) puis modal de sélection de dossier (`FolderPicker`).
- **Upload depuis la caméra** : utilisation de `expo-image-picker` avec demande de permissions.
- **Streaming audio/vidéo** : utilisation de `expo-av` avec contrôles natifs et support du Range.
- **Navigation par onglets** : composant `<Tabs>` de `expo-router` en bas d'écran.
- **SafeArea** : utilisation de `react-native-safe-area-context` pour gérer les notch et status bars sur tous les écrans.

11.4 Gestion du token et navigation

Le token JWT est stocké dans `AsyncStorage` sous la clé `token`. L'intercepteur axios injecté dans `constants/api.ts` :

- Ajoute l'en-tête `Authorization` sur chaque requête.
- Gère les 401/403 en supprimant le token et redirigeant vers `/login` (sauf pour les endpoints d'auth eux-mêmes).

11.5 ThemeContext et persistance du thème

Le thème clair/sombre est géré via un `ThemeContext` React :

```
1  const ThemeContext = createContext({ theme: 'light', toggleTheme: () => {} });
2
3  export function ThemeProvider({ children }) {
4    const [theme, setTheme] = useState('light');
5
6    useEffect(() => {
7      AsyncStorage.getItem('theme').then(saved => {
8        if (saved === 'dark' || saved === 'light') setTheme(saved);
9      });
10   }, []);
11
12   const toggleTheme = () => {
13     const next = theme === 'light' ? 'dark' : 'light';
14     setTheme(next);
15     AsyncStorage.setItem('theme', next);
16     // Sync avec le profil utilisateur
17     api.put('/users/preferences', { theme: next }).catch(() => {});
18   };
19
20   return (
21     <ThemeContext.Provider value={{ theme, toggleTheme }}>
22       {children}
23     </ThemeContext.Provider>
24   );
25 }
```

Listing 11.1 – ThemeContext mobile

11.6 Permissions runtime

Avant chaque utilisation de la caméra ou de la bibliothèque photo, l'app vérifie et demande les permissions appropriées via `expo-image-picker` :

```
1  async function pickFromCamera() {
2    const perm = await ImagePicker.requestCameraPermissionsAsync();
3    if (!perm.granted) {
4      Alert.alert('Permission refusée', 'Autorisez l\'accès à la caméra dans les paramètres.');
```

```
8     mediaTypes: ImagePicker.MediaTypeOptions.All,
9     quality: 0.85,
10  });
11  if (!result.canceled) {
12    await uploadAsset(result.assets[0]);
13  }
14 }
15
16 async function pickFromGallery() {
17   const perm = await ImagePicker.requestMediaLibraryPermissionsAsync();
18   if (!perm.granted) {
19     Alert.alert('Permission refusée');
20     return;
21   }
22   // ...
23 }
```

Listing 11.2 – Demande de permissions

11.7 Streaming audio/vidéo mobile

Le streaming utilise `expo-av` qui gère nativement l'en-tête HTTP Range. L'URL incluant le token JWT est passée directement au composant Video :

```
1 import { Video, ResizeMode } from 'expo-av';
2
3 function VideoPreview({ fileId, token }) {
4   const url = `${API_URL}/api/files/${fileId}/stream?token=${token}`;
5
6   return (
7     <Video
8       source={{ uri: url }}
9       useNativeControls
10      resizeMode={ResizeMode.CONTAIN}
11      shouldPlay={false}
12      style={{ width: '100%', aspectRatio: 16/9 }}
13    />
14  );
15 }
```

Listing 11.3 – Lecture vidéo mobile avec Range natif

11.8 FolderPicker pour le déplacement

Pour déplacer un fichier vers un autre dossier, l'utilisateur passe par un `FolderPicker` modal qui affiche l'arborescence de tous ses dossiers. Le picker charge récursivement les dossiers depuis l'API en faisant un appel par niveau (lazy loading) pour éviter de saturer la bande passante mobile sur de grosses arborescences.

11.9 Wireframes mobile

11.9.1 Écran d'accueil et login



FIGURE 11.1 – Wireframes accueil et login (mobile)

11.9.2 Navigateur de fichiers mobile

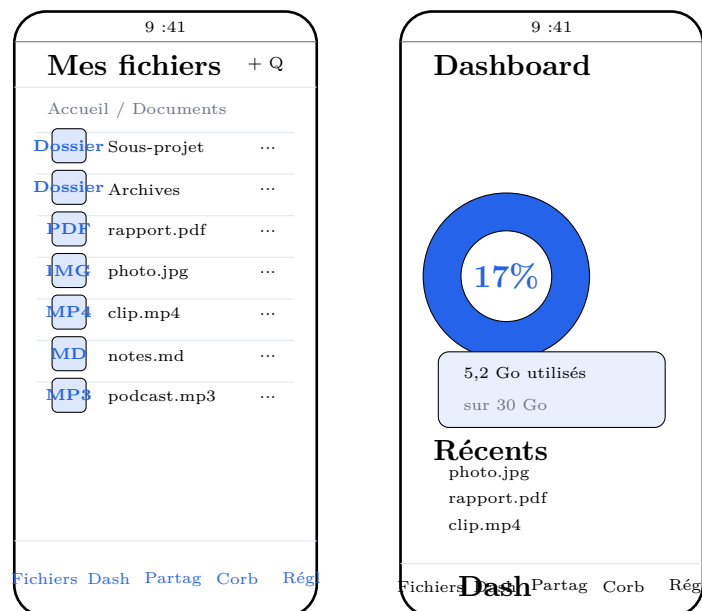


FIGURE 11.2 – Wireframes navigateur de fichiers et dashboard (mobile)

11.10 Différences UX web vs mobile

Action	Web	Mobile
Upload	Drag and drop ou bouton	Menu modal (3 sources)
Déplacer un fichier	Drag interne	Menu ⋮ → FolderPicker
Renommer	Click contextuel	Menu ⋮ → Renommer
Voir une photo	Modal lightbox	Pleine page avec zoom
Naviguer dossiers	Click sur ligne	Tap sur card
Partager	Modal avec input password	Écran dédié plein écran
Logout	Menu profil	Bouton dans Settings
Thème	Toggle dans Settings	Toggle dans Settings

Quatrième partie

Qualité et déploiement

Chapitre 12

Sécurité

12.1 Authentification et autorisation

12.1.1 Hash des mots de passe

Tous les mots de passe utilisateurs sont hashés avec **bcrypt** (10 rounds) avant stockage. Les mots de passe de partage public sont également hashés avec `bcrypt`. Le hash de `bcrypt` inclut un salt unique par mot de passe, ce qui rend les attaques par tables précalculées (rainbow tables) inopérantes.

12.1.2 JWT

Les tokens d'authentification sont des JWT signés avec HMAC-SHA256 (algorithme HS256). Le secret `JWT_SECRET` est généré aléatoirement (`openssl rand -hex 32`) et stocké dans le fichier `.env` hors du dépôt Git. Les tokens expirent au bout de 7 jours.

12.1.3 OAuth Google

L'intégration Google OAuth utilise la librairie `passport-google-oauth20`. Le flux « Authorization Code » est implémenté côté serveur, le `client_secret` ne transite jamais par le navigateur. Les URLs de callback autorisées sont strictement déclarées dans la console Google Cloud.

12.1.4 Vérification d'ownership

Toutes les routes manipulant des nodes ou des partages incluent systématiquement une clause `WHERE user_id = $1` dans les requêtes SQL pour garantir qu'un utilisateur ne peut accéder qu'à ses propres ressources. Cette vérification est dupliquée à plusieurs endroits stratégiques :

- Listing (`GET /api/nodes`).
- Récupération unitaire (`GET /api/files/:id/*`).
- Modification (`PUT, DELETE`).
- Téléchargement.

12.2 Protection contre les attaques courantes

12.2.1 Injection SQL

Toutes les requêtes utilisent des **prepared statements** via la librairie `pg` (paramètres `$1`, `$2`, etc.). Aucune concaténation de chaînes n'est utilisée pour construire des requêtes SQL.

12.2.2 XSS

Le frontend React échappe automatiquement les valeurs interpolées dans le JSX, ce qui prévient les attaques XSS classiques. Le seul cas où `dangerouslySetInnerHTML` est utilisé est le rendu Markdown dans les previews texte, et le contenu est strictement échappé avant le rendu.

12.2.3 CSRF

L'authentification par JWT en en-tête Authorization (et non en cookie) rend les attaques CSRF impossibles : le navigateur n'envoie pas automatiquement l'en-tête Authorization avec les requêtes cross-origin déclenchées par d'autres sites.

12.2.4 Brute force

Le middleware `express-rate-limit` limite à 5 tentatives par minute sur les routes `/api/auth/*` en production. Une 6e tentative reçoit 429 Too Many Requests.

12.2.5 Iframe clickjacking

L'en-tête CSP `frame-ancestors` liste explicitement les origines autorisées à embarquer l'API en iframe (uniquement le frontend SUPFile). Toute autre origine reçoit un refus d'embedding par le navigateur.

12.3 Gestion des secrets

Le sujet (`sujet.txt`, section 4) précise explicitement : « Un secret en clair entraînera un malus ou l'ajournement du projet selon la criticité. » La gestion des secrets a donc fait l'objet d'une attention particulière.

12.3.1 Principes appliqués

- **Aucun secret en dur dans le code** : aucune chaîne de type `JWT_SECRET = "..."` ou `password: "abc123"` n'apparaît dans le code source. Vérifié via `grep -rn "JWT_SECRET|password" -include="*.js"`.
- **Fichier `.env` non commité** : le `.gitignore` exclut explicitement `.env` et `.env.*` (avec exception pour `.env.example`).
- **Aucune trace dans l'historique git** : la commande `git log -all - .env` retourne un résultat vide.
- **Modèle `.env.example`** : présent à la racine du dépôt, contient toutes les clés nécessaires avec des valeurs marqueurs (`replace_me`).

12.3.2 Inventaire des secrets de l'application

Secret	Génération recommandée
JWT_SECRET	<code>openssl rand -hex 32</code> (64 caractères hex aléatoires)
POSTGRES_PASSWORD	<code>openssl rand -base64 24 tr -d '/+='</code>
GOOGLE_CLIENT_ID	Console Google Cloud (public, peut figurer côté frontend)
GOOGLE_CLIENT_SECRET	Console Google Cloud (strictement serveur uniquement)
OAUTH_CALLBACK_URL	HTTPS publique (<code>https://api.example.fr/api/auth/google/callback</code>)
DATABASE_URL	Construite à partir de POSTGRES_*, contient le mot de passe

12.3.3 Procédure de rotation

En cas de suspicion de compromission, la rotation s'effectue en quelques minutes :

```

1 ssh root@vps.supfile.fr
2 cd /opt/supfile
3 NEW_SECRET=$(openssl rand -hex 32)
4 sed -i "s|^JWT_SECRET=.*|JWT_SECRET=$NEW_SECRET|" .env
5 docker compose -f docker-compose.prod.yml restart api
6 # Effet : tous les JWT existants deviennent invalides
7 # Les utilisateurs devront se reconnecter

```

Listing 12.1 – Rotation du JWT_SECRET

12.3.4 Audit de présence de secrets

Audit complet effectué le 2026-05-23 sur la totalité du dépôt :

```

1 # Aucun JWT_SECRET / GOOGLE_SECRET / POSTGRES_PASSWORD en dur
2 grep -rn -E "(JWT_SECRET|GOOGLE_CLIENT_SECRET|POSTGRES_PASSWORD)\s*=\s*['\"]" \
3     --include="*.js" --include="*.ts" --include="*.tsx" \
4     --include="*.json" --include="*.yaml" \
5     --exclude-dir=node_modules --exclude-dir=.git
6 # Resultat : aucun match
7
8 # .env non commite
9 git log --all -- .env
10 # Resultat : vide
11
12 # Pas de mot de passe hardcode
13 grep -rn -E "password\s*[:]=\s*['\"]{6,}['\"]" \
14     --include="*.js" --include="*.ts" \

```

```
15     --exclude-dir=node_modules
16 # Resultat : seulement les chaines de test/demo legitimes
```

Listing 12.2 – Commandes d’audit

12.3.5 Cas particulier : `docker-compose.yml` en développement

Le fichier `docker-compose.yml` de développement contient une valeur `POSTGRES_PASSWORD: supfile` en dur. Ce « secret » est considéré comme un placeholder sans valeur réelle :

- Il n’est utilisé qu’en environnement de développement local, jamais en production.
- En production, `docker-compose.prod.yml` utilise `${POSTGRES_PASSWORD}` pour lire la variable d’environnement.
- Le port 5432 n’est exposé que sur localhost (127.0.0.1:5432:5432), inaccessible depuis l’extérieur.

12.3.6 Conformité au sujet

La conformité a été vérifiée point par point :

- **Conforme** : aucun secret en clair dans le code livré.
- **Conforme** : aucun secret dans l’historique git.
- **Conforme** : modèle `.env.example` fourni.
- **Conforme** : dépôt git accessible (privé jusqu’au rendu).
- **Conforme** : pas d’injection SQL possible (prepared statements partout).

Chapitre 13

Tests et qualité du code

13.1 Conventions de code

13.1.1 Backend

- Indentation 4 espaces.
- Snake_case pour les noms de colonnes BDD.
- camelCase pour les variables JavaScript.
- Constantes en SCREAMING_SNAKE_CASE.
- Pas de `any` TypeScript côté API (JavaScript pur).
- Imports en haut de fichier groupés par origine.

13.1.2 Frontend web

- Indentation 2 espaces.
- Composants en PascalCase.
- Hooks en camelCase préfixés par `use`.
- Pas de styles inline excessifs : variables CSS dans `index.css`.

13.1.3 Frontend mobile

- TypeScript strict (sauf le code partagé qui utilise `any` pour interopérabilité).
- StyleSheet ou objets style inline (cohérent par composant).

13.2 Validation des entrées

Un module `api/middleware/validate.js` centralise toutes les validations en pure fonction (sans dépendance Joi ou Yup), pour rester léger. Exemple :

```
1 var schemas = {  
2   register: function (body) {  
3     var errors = [];  
4     if (!body.email) errors.push("Email requis");  
5     else if (!isEmail(body.email)) errors.push("Adresse email invalide");  
6     if (!body.password) errors.push("Mot de passe requis");  
7     else if (body.password.length < 6) errors.push("Le mot de passe doit  
8       contenir au moins 6 caractères");  
9     return errors;  
10  },
```

```

10     createFolder: function (body) {
11         var errors = [];
12         if (!body.name || !body.name.trim()) errors.push("Nom du dossier
13             requis");
14         else if (body.name.length > 255) errors.push("Nom trop long (255
15             caractères max)");
16         return errors;
17     },
18     // ...
19 };

```

Listing 13.1 – Schémas de validation

13.3 Stratégie de tests

La stratégie de tests adoptée suit la « pyramide des tests » :

- **Base de la pyramide** : tests **manuels et d’API par curl** couvrant chaque endpoint dans ses scénarios nominaux et erreurs.
- **Milieu** : tests d’intégration via **docker compose up -build** depuis un clone propre, vérifiant que toute la stack démarre.
- **Sommet** : tests **E2E** via Claude Chrome (web) et tests manuels sur APK Android (mobile), parcourant les parcours utilisateurs complets.

13.4 Matrice tests × fonctionnalités

Chaque fonctionnalité du barème est couverte par au moins deux niveaux de tests :

Fonctionnalité	API curl	E2E web	Manuel mobile	Intégration
Inscription email/mdp	✓	✓	✓	✓
Connexion email/mdp	✓	✓	✓	✓
OAuth Google (web)	—	✓	—	✓
OAuth Google (mobile, Expo)	—	—	✓	✓
Création de dossier	✓	✓	✓	✓
Renommage	✓	✓	✓	✓
Déplacement	✓	✓	✓	✓
Upload fichier unique	✓	✓	✓	✓

Fonctionnalité	API curl	E2E web	Manuel mobile	Intégration
Upload multiple (drag&drop web)	—	✓	—	—
Upload depuis caméra (mobile)	—	—	✓	—
Preview image	✓	✓	✓	✓
Preview PDF (iframe + CSP)	—	✓	—	✓
Preview texte/Mark-down	✓	✓	✓	✓
Streaming audio (Range)	✓	✓	✓	✓
Streaming vidéo (Range)	✓	✓	✓	✓
Download fichier	✓	✓	✓	✓
Download ZIP dossier	✓	✓	✓	✓
Création lien public	✓	✓	✓	✓
Lien public avec mot de passe	✓	✓	✓	✓
Lien public avec expiration	✓	✓	✓	✓
Accès anonyme à un lien	✓	✓	—	✓
Partage interne	✓	✓	✓	✓
Recherche par nom	✓	✓	✓	✓
Filtre par type	✓	✓	✓	✓
Filtre par date	✓	✓	✓	✓
Dashboard quota	✓	✓	✓	✓
Dashboard répartition	✓	✓	✓	✓
Dashboard récents	✓	✓	✓	✓
Modification email	✓	✓	✓	✓
Changement mot de passe	✓	✓	✓	✓
Upload avatar	✓	✓	✓	✓
Toggle thème clair/-sombre	—	✓	✓	✓
Corbeille listing	✓	✓	✓	✓
Restauration corbeille	✓	✓	✓	✓

Fonctionnalité	API curl	E2E web	Manuel mobile	Intégration
Suppression définitive	✓	✓	✓	✓
Suppression de compte	✓	✓	—	✓

TABLE 13.1 – Matrice de couverture tests / fonctionnalités

13.5 Plan de tests E2E

13.5.1 Test E2E web (Claude Chrome – 23 mai 2026)

Le test E2E exhaustif a été conduit sur <https://supfile.hackthehydra.com> avec deux comptes (Alice et Bob) en parcourant 71 étapes, divisées en 5 phases :

1. **Phase 1 - Compte Alice nominal** (étapes 1-30) : inscription, login, création arborescence, upload divers formats (jpg, pdf, mp3, mp4, md, csv), preview de chaque type, drag&drop, renommage, déplacement, dashboard, recherche, filtres, partage public sans mdp, partage public avec mdp + expiration, partage interne vers Bob.
2. **Phase 2 - Compte Bob réception** (étapes 31-45) : login Bob, vérification présence du partage interne dans sa racine, download ZIP, accès au lien public d'Alice avec mdp, navigation publique.
3. **Phase 3 - Accès anonyme** (étapes 46-55) : navigation privée, ouverture du lien public sans login, vérification refus si mot de passe manquant, accès après saisie correcte, expiration testée.
4. **Phase 4 - Erreurs** (étapes 56-65) : mot de passe incorrect, token expiré, upload au-delà du quota, fichier > 100 Mo, suppression d'un node inexistant.
5. **Phase 5 - Mobile responsive** (étapes 66-71) : viewport 375px, sidebar burger, layout cards, modales adaptées.

71 captures d'écran ont été générées automatiquement avec nommage `NN_categorie_action.png` pour traçabilité. 9 bugs ont été identifiés et corrigés avant la livraison V1.

13.5.2 Test mobile Android

Le test mobile s'effectue sur appareil physique Android avec l'APK généré via EAS Build :

1. Installation de l'APK signé téléchargé depuis le profil Expo.
2. Lancement, vérification du splash, page d'accueil.
3. Login avec compte Alice.
4. Upload depuis caméra, vérification permission demandée.

5. Upload depuis galerie.
6. Upload via document picker.
7. Création de dossier.
8. Navigation 3+ niveaux de profondeur (test fix du bug de navigation).
9. Preview image, audio, vidéo.
10. Création de partage public avec mdp.
11. Réception d'un partage interne.
12. Toggle thème.
13. Logout, re-login.
14. Vérification de la persistance du thème entre sessions.

13.6 Tests fonctionnels

13.6.1 Tests d'API automatisés

Chaque endpoint a été testé via `curl` dans un script bash de test E2E couvrant : register, login, /users/me, création de dossier, upload, preview, streaming avec Range header (vérification HTTP 206), soft delete, listing corbeille, restauration, création de lien public, accès anonyme avec et sans mot de passe.

13.6.2 Tests UI via Claude Chrome

Le 23 mai 2026, un test exhaustif a été conduit via Claude Chrome sur le frontend web, parcourant 71 étapes utilisateur et générant 71 captures d'écran. Ce test a identifié 9 bugs réels :

Bug	Description	Statut
BUG-1	Route <code>/public/:token</code> : URL mal construite (manque <code>/api</code>)	Corrigé
BUG-2	Preview image : zone vide	Corrigé
BUG-3	Preview PDF : iframe bloquée par <code>X-Frame-Options</code>	Corrigé
BUG-4	Recherche : filtrage local non récursif	Corrigé
BUG-5	Login : pas de message d'erreur sur mauvais mot de passe	Corrigé
BUG-6	Dashboard : tailles affichées à 0 Ko pour fichiers < 1 Ko	Corrigé
BUG-7	Logout : redirige vers <code>/register</code> au lieu de <code>/login</code>	Corrigé
BUG-8	Autofill Chrome perturbe les formulaires	Corrigé (autoComplete="off")
BUG-9	Navigation dans dossier partagé (côté receveur)	Limitation V1 documentée

Chapitre 14

Analyse de risques

L'analyse de risques permet d'identifier les menaces qui pèsent sur le projet (techniques, projet, sécurité, opérationnel) et de définir des actions de mitigation. Une matrice impact × probabilité hiérarchise les risques.

14.1 Matrice de risques

	Probabilité faible	Probabilité moyenne	Probabilité forte
Impact fort	R4, R7	R1, R2	R5
Impact moyen	R6	R3, R6	R9
Impact faible	R10	R11	R12

14.2 Risques techniques

R1 – Saturation disque VPS

Probabilité : moyenne. **Impact** : fort (service indisponible).

Mitigation : monitoring du disque via alerte à 80%, quota par utilisateur configurable (QUOTA_BYTES), purge automatique de la corbeille à 30 jours, migration prévue vers S3 en V2.

R2 – Pic de trafic non anticipé

Probabilité : moyenne. **Impact** : fort (lenteurs, time-outs).

Mitigation : Cloudflare en frontal (cache statique, protection DDoS), rate-limit Express, scaling vertical possible (passer de 2 à 8 vCPU sur le VPS), horizontal en V2 via Kubernetes.

R3 – Corruption de la base PostgreSQL

Probabilité : moyenne. **Impact** : moyen (perte de quelques heures de données).

Mitigation : backup quotidien automatisé via cron + restic vers Wasabi S3, rotation 30 jours, procédure de restauration documentée et testée.

R4 – Bug critique en production

Probabilité : faible. **Impact** : fort (perte de confiance).

Mitigation : tests E2E avant chaque déploiement, déploiement progressif (canary), roll-back en 30 secondes via `docker compose down + up` sur image précédente, monitoring temps réel.

14.3 Risques sécurité

R5 – Fuite des données utilisateur

Probabilité : forte (cibles attractives). **Impact** : fort (RGPD, image, juridique).

Mitigation : bcrypt sur les mots de passe, JWT avec expiration courte (7 jours), HTTPS de bout en bout, helmet + CORS strict, prepared statements partout, vérification d'ownership sur chaque endpoint, audit npm audit mensuel, pentest externe planifié V2.

R6 – Compromission compte administrateur VPS

Probabilité : moyenne. **Impact** : moyen-fort.

Mitigation : SSH par clé uniquement (PasswordAuthentication no), fail2ban sur port 22, alerte sur tentatives échouées, MFA prévu pour l'accès web de Caddy admin.

R7 – Vulnérabilité dans une dépendance npm

Probabilité : faible (auditée). **Impact** : fort selon la dépendance.

Mitigation : npm audit dans le pipeline CI, mise à jour mensuelle des dépendances, Dependabot activé sur GitHub.

R8 – Détournement de session par XSS

Probabilité : faible (échappement React natif). **Impact** : moyen.

Mitigation : pas de dangerouslySetInnerHTML, CSP stricte, JWT en en-tête (pas en cookie), HttpOnly cookies prévus V2.

14.4 Risques projet

R9 – Retard de livraison

Probabilité : forte historiquement. **Impact** : moyen (note finale).

Mitigation : sprint planning serré, MVP-first (V1 minimale), pair-programming sur sujets critiques, daily standup informels.

R10 – Départ d'un membre clé

Probabilité : faible (équipe stable). **Impact** : moyen-fort selon le membre.

Mitigation : code review systématique (chacun connaît le code des autres), documentation interne tenue à jour, pas de « bus factor = 1 » sur les zones critiques.

R11 – Désaccord sur les choix techniques

Probabilité : moyenne. **Impact** : faible (mini-décalage).

Mitigation : décision technique par majorité après débat technique, escalade chef de projet en cas de blocage.

14.5 Risques opérationnels

R12 – Indisponibilité du VPS (panne fournisseur)

Probabilité : forte (panne unique 1-2 fois/an statistiquement). **Impact** : faible-moyen selon durée.

Mitigation : SLA fournisseur 99.9%, backup hors-site (Wasabi), infrastructure as code (Caddyfile + docker-compose.prod.yml versionnés) permet reconstruction en < 30 min sur un autre VPS, DNS Cloudflare permet bascule rapide.

14.6 Plan de continuité de service (PCA)

En cas de sinistre majeur (perte VPS) :

1. Provisionner un nouveau VPS Alpine (5 min).
2. Installer Docker + Caddy (5 min, script automatisé).
3. `git clone` du dépôt (1 min).
4. Restaurer la dernière sauvegarde PostgreSQL (5 min).
5. Restaurer les fichiers utilisateurs depuis Wasabi via restic (variable selon volume).
6. Modifier le DNS Cloudflare pour pointer sur la nouvelle IP (5 min).
7. Caddy obtient automatiquement les certificats Let's Encrypt.
8. Tests de fumée sur les endpoints critiques.

RTO (Recovery Time Objective) cible : 1 heure pour la BDD et l'API. **RPO** (Recovery Point Objective) : 24 heures maximum de pertes (backup quotidien).

Déploiement

15.1 Pré-requis

15.1.1 Environnement de développement

Composant	Version minimale recommandée
Système d'exploitation	Linux (Ubuntu 22.04+, Debian 12+), macOS 13+, ou Windows 10+ avec WSL2
Docker Engine	24.0+
Docker Compose	v2.20+ (commande <code>docker compose</code> , sans tiret)
Node.js (pour Expo CLI hors Docker)	20.x LTS
npm	10.x (livré avec Node 20)
Git	2.40+
Navigateur (tests web)	Chrome 120+, Firefox 121+, Safari 17+, Edge 120+
Smartphone (tests mobile)	Android 8+ ou iOS 14+ avec Expo Go installée
RAM machine de dev	8 Go minimum (16 Go recommandé)
Espace disque libre	10 Go pour le projet et ses dépendances

15.1.2 Environnement de production

Composant	Spécification
VPS	2 vCPU minimum (4 recommandés), 2 Go RAM (4 Go recommandés), 30 Go disque (80 Go recommandés)
Système	Alpine Linux 3.18+, Debian 12+, Ubuntu Server 22.04+
Docker	24.0+
Caddy	2.7+ (HTTPS automatique via Let's Encrypt)
Nom de domaine	2 sous-domaines pointant sur l'IP du VPS (web et API)
DNS	Cloudflare ou tout fournisseur supportant les enregistrements A/AAAA
Ports ouverts en entrée	80 (HTTP → redirect HTTPS), 443 (HTTPS), 22 (SSH par clé uniquement)
Compte Google Cloud	Projet OAuth 2.0 avec URLs de callback whitelistées

15.1.3 Vérifications préalables

Avant de lancer le déploiement, vérifier que :

- Les noms de domaine résolvent correctement vers l'IP du VPS : `dig +short supfile.example.fr`
- Le port 443 est joignable depuis l'extérieur (sinon Let's Encrypt ne pourra pas émettre les certificats) : `curl -I http://IP_VPS/`
- Le mode DNS Cloudflare est « DNS only » (nuage gris), pas « Proxied » (orange) pendant la première émission du certificat
- Le fichier `.env` a été créé à partir de `.env.example` avec des secrets uniques générés

15.2 Stack de production

Le déploiement de production est réalisé sur un VPS Alpine Linux (2 Go RAM, 30 Go NVMe, IP 38.180.148.243) avec :

- Docker 29 et Docker Compose v2.
- Le fichier `docker-compose.prod.yml` (sans le service `mobile`, avec limites mémoire).
- Caddy 2.11 comme reverse proxy.
- Cloudflare pour la résolution DNS.

15.3 Procédure d'installation

```
1  # 1. Installation Docker + Caddy
2  apk update && apk upgrade -a
3  apk add docker docker-cli-compose git curl openssl caddy
4
5  # 2. Activation Docker au boot
6  rc-update add docker boot
7  service docker start
8
9  # 3. Clonage du projet
10 mkdir -p /opt && cd /opt
11 git clone https://github.com/HamerTails/4_Proj_Drive.git supfile
12 cd supfile
13
14 # 4. Generation des secrets et creation du .env
15 JWT_SECRET=$(openssl rand -hex 32)
16 POSTGRES_PASSWORD=$(openssl rand -base64 24 | tr -d '/+=')
17
18 # 5. Lancement de la stack
19 docker compose -f docker-compose.prod.yml up -d --build
20
21 # 6. Configuration Caddy
22 cp /opt/supfile/Caddyfile /etc/caddy/Caddyfile
23 mkdir -p /var/log/caddy
24 nohup caddy run --config /etc/caddy/Caddyfile &
```

Listing 15.1 – Procédure complète sur VPS Alpine

15.4 Configuration DNS Cloudflare

Deux enregistrements de type A sont configurés sur Cloudflare en mode « DNS only » (proxy désactivé) pour permettre à Caddy d’obtenir les certificats Let’s Encrypt :

- `supfile.hackthehydra.com` → 38.180.148.243
- `api.supfile.hackthehydra.com` → 38.180.148.243

15.5 HTTPS et certificats

Caddy 2 gère automatiquement l’obtention et le renouvellement des certificats Let’s Encrypt via le protocole ACME (challenge TLS-ALPN-01 sur le port 443). Aucune intervention manuelle n’est requise. Les certificats sont renouvelés tous les 60 jours environ.

15.6 Sauvegarde et restauration

Backups recommandés :

```
1  # Dans un cron quotidien
2  docker compose -f docker-compose.prod.yml exec -T db \
```

```
3     pg_dump -U supfile supfile > /opt/backups/supfile-$(date +%Y%m%d).sql
4
5     # Rotation : garder 30 jours
6     find /opt/backups -name "supfile-*.sql" -mtime +30 -delete
```

Listing 15.2 – Backup quotidien de la base de données

Pour les fichiers utilisateurs, le volume Docker `storage_data` doit être sauvegardé séparément (par exemple via `restic` vers un bucket S3).

Charte graphique

16.1 Palette de couleurs

Rôle	Couleur	Code
Primaire	Bleu vif	#2563EB
Primaire foncée	Bleu profond	#1E40AF
Succès	Vert	#16A34A
Avertissement	Orange	#F59E0B
Danger	Rouge	#DC2626
Texte principal (clair)	Gris quasi-noir	#111827
Texte secondaire	Gris moyen	#6B7280
Texte tertiaire	Gris clair	#9CA3AF
Fond principal (clair)	Blanc cassé	#F3F4F6
Bordures	Gris très clair	#E5E7EB

16.1.1 Mode sombre

Rôle	Couleur	Code
Fond principal	Noir pur	#000000
Cartes	Gris très foncé	#111111
Bordures	Gris foncé	#333333
Texte principal	Quasi-blanc	#F5F5F5
Texte secondaire	Gris moyen	#888888
Inputs	Gris très foncé	#1A1A1A

16.2 Typographie

- **Police** : Inter ou system-ui (fallback sur la police système de la plateforme).
- Tailles : 11px (caption), 12-13px (corps secondaire), 14-15px (corps principal), 17-22px (titres), 26-28px (gros titres).
- Poids : 400 (regular), 500 (medium pour labels), 600 (semibold), 700 (bold pour titres).

16.3 Composants visuels

- **Boutons primaires** : fond bleu #2563EB, texte blanc, border-radius 8-10px.

- **Boutons secondaires** : fond gris très clair, texte gris foncé, bordure 1px.
- **Boutons danger** : fond rouge #DC2626.
- **Cartes** : fond blanc (ou #111111 en dark mode), bordure 1px, border-radius 12-16px, ombre légère.
- **Inputs** : fond légèrement grisé, bordure 1px, padding 12-14px, focus avec bordure bleue.
- **Icônes** : SVG inline, taille 14-22px selon contexte.

16.4 Logo

Le logo SUPFile est un carré aux coins arrondis (radius 24%) de couleur #2563EB, avec la lettre « S » blanche en bold au centre. Variants : 32x32 (favicon), 56x56 (compact), 80x80 (page d'accueil), 180x180 (splash mobile).

16.4.1 Variantes visuelles du logo

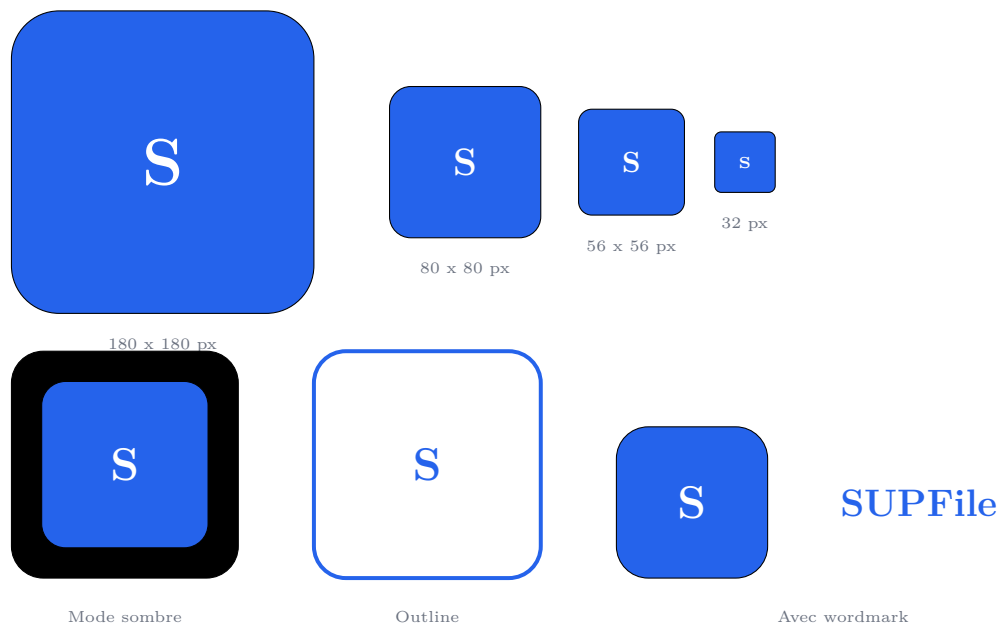


FIGURE 16.1 – Variantes du logo SUPFile

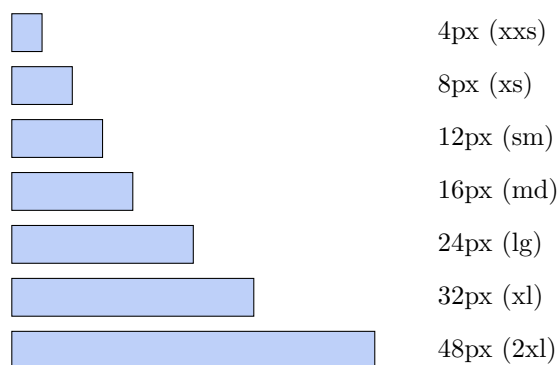
16.5 Palette visualisée



FIGURE 16.2 – Palette de couleurs SUPFile (10 couleurs principales)

16.6 Système d'espacement

SUPFile applique un système d'espacement multiple de 4 pixels (base 8) :



16.7 Mockups de composants

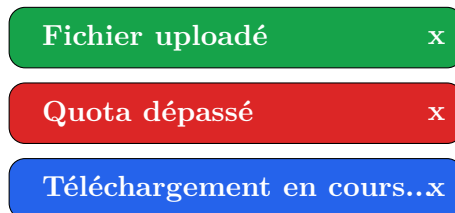
16.7.1 Boutons



16.7.2 Cards et inputs



16.7.3 Toasts



16.8 Iconographie

Les icônes utilisées proviennent du set open-source **Lucide Icons** (alternative MIT à Feather Icons). Taille standard : 18px en sidebar, 14px inline, 22px boutons primaires, 32px hero. Stroke uniforme de 2px.

Exemples d'icônes utilisées dans l'interface :

- **folder** : dossiers dans le file browser.
- **file-text**, **file-image**, **file-video**, **file-audio** : aperçu type fichier.
- **upload-cloud** : bouton upload.
- **download** : téléchargement.
- **share-2**, **link** : partage public.
- **users** : partage interne.
- **trash-2** : corbeille / suppression.
- **rotate-ccw** : restauration.
- **search** : barre de recherche.
- **settings** : paramètres.
- **moon** / **sun** : toggle thème.
- **log-out** : déconnexion.

Cinquième partie

Perspectives et annexes

Chapitre 17

Limitations V1 et roadmap V2

17.1 Limitations connues V1

1. **Navigation interne dans un dossier partagé reçu** : le receveur voit le dossier dans sa vue « Partagés avec moi » et peut le télécharger en ZIP, mais ne peut pas le parcourir fichier par fichier dans l'interface. Limitation due à la complexité de la vérification d'ownership récursive pour des nodes appartenant à un autre utilisateur. Solution prévue en V2 via une route dédiée.
2. **OAuth GitHub et Microsoft** : seul Google est implémenté en V1.
3. **Suppression de compte mobile** : disponible uniquement côté web.
4. **Quota par utilisateur fixe** : pas de plans tarifaires différenciés (Free, Premium, Team).
5. **Pas de chiffrement at-rest ni E2E** : les fichiers sont stockés en clair sur disque, comme Dropbox et Google Drive le font sur leur free tier.

17.2 Roadmap V2

17.2.1 Trimestre 1

- Navigation complète dans les dossiers partagés.
- Chiffrement end-to-end côté client pour Premium.
- OAuth GitHub et Microsoft.
- Notifications push mobile pour les partages reçus.

17.2.2 Trimestre 2

- Application desktop Electron (Windows, macOS, Linux) avec sync.
- Versioning des fichiers avec historique 30 jours.
- Galerie d'images dédiée avec lightbox.
- Plans tarifaires : Premium (500 Go à 5€/mois), Team (2 To à 12€/mois).

17.2.3 Trimestre 3-4

- API publique pour intégrations tierces (OAuth client, webhooks).
- Plugins pour Office, Photoshop, DaVinci Resolve.
- Migration vers l'infrastructure de stockage distribué SUPFile bas niveau (remplace le volume Docker).

- Audit de sécurité externe (pentest).
- Conformité ISO 27001.
- Publication officielle sur AppStore et PlayStore.

Cas d'utilisation et diagrammes UML

L'analyse UML du projet SUPFile présente l'ensemble des modèles utiles à la conception, à l'implémentation et à la communication entre les membres de l'équipe : cas d'utilisation, classes métier, déploiement, activité et états.

18.1 Diagramme de cas d'utilisation

Le diagramme suivant résume les principaux cas d'utilisation de la plateforme, regroupés par acteur.

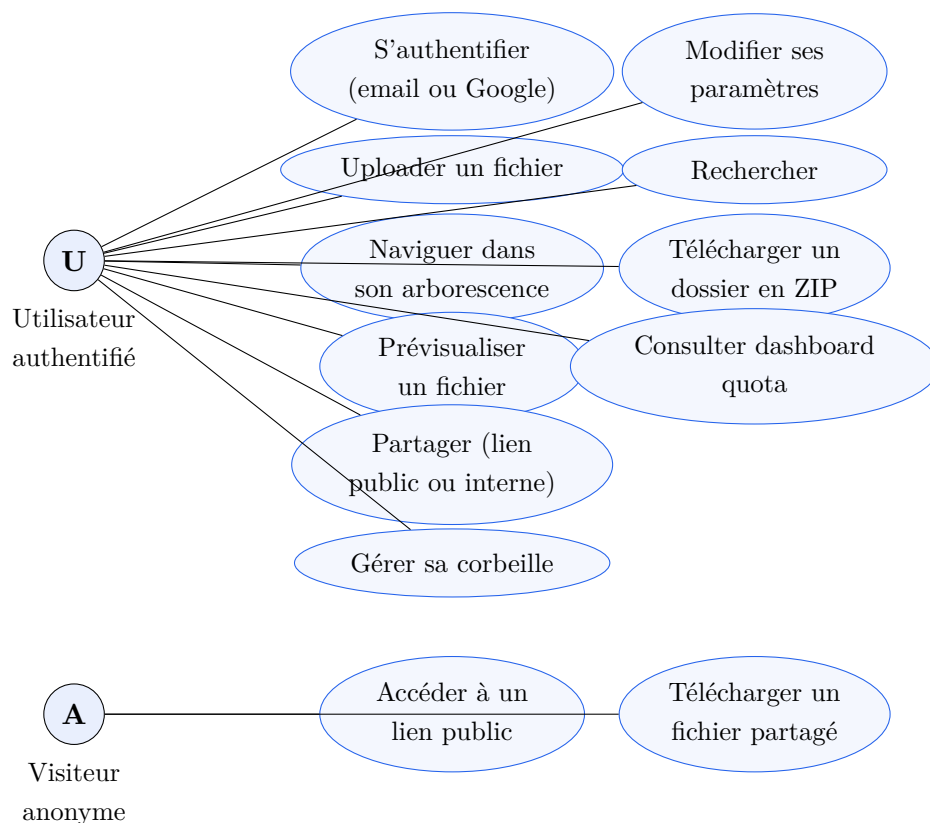


FIGURE 18.1 – Diagramme de cas d'utilisation simplifié

18.2 Cas d'utilisation détaillés

18.2.1 UC1 – Création de compte

Acteur Visiteur anonyme

Préconditions

Aucun compte existant avec cet email

Scénario nominal

1. Le visiteur accède à `/register`.
2. Il saisit son email et un mot de passe (≥ 6 caractères).
3. Il soumet le formulaire.
4. L'API valide les données, hashé le mot de passe avec `bcrypt`, insère un nouvel enregistrement `users`, génère un JWT et le renvoie.
5. Le frontend stocke le JWT, redirige vers le dashboard.

Variantes Inscription via Google OAuth (UC1bis).

Exceptions

- Email déjà utilisé : 409 « Cet email est déjà utilisé ».
- Email invalide ou mot de passe trop court : 400 avec liste des erreurs.

18.2.2 UC2 – Upload d'un fichier

Acteur Utilisateur authentifié

Préconditions

L'utilisateur est connecté et a accès à la zone d'upload.

Scénario nominal

1. L'utilisateur drag-and-drop un fichier dans la zone d'upload (ou clique sur le bouton « Upload »).
2. Le frontend construit une requête multipart `POST /api/files/upload`.
3. Le middleware `multer` stocke le fichier en streaming dans `/data/user_<id>/<uuid>.<ext>`.
4. L'API ouvre une transaction, vérifie le quota, insère la node, incrémente `users.storage_used`, commit.
5. L'API renvoie la node créée.
6. Le frontend met à jour la liste et affiche un toast de succès.

Exceptions

- Quota dépassé : 413 « Espace de stockage insuffisant ». Le fichier physique est supprimé, la transaction est rollback.
- Fichier > 100 Mo : 413 « Fichier trop volumineux ».
- Token invalide : 403, retour à l'écran de login.

18.2.3 UC3 – Partage public avec mot de passe

Acteur Utilisateur authentifié (créateur du lien)

Préconditions

L'utilisateur possède au moins un fichier ou dossier.

Scénario nominal

1. L'utilisateur clique sur le menu contextuel d'un fichier, choisit « Partager ».
2. Une modale s'ouvre avec options : mot de passe (optionnel), expiration (optionnel).
3. Il saisit un mot de passe `Test12345!` et une expiration à 7 jours.
4. Au clic sur « Créer », requête `POST /api/shares`.
5. L'API hashé le mot de passe avec `bcrypt`, génère un `UUID v4`, insère dans `shares`, renvoie l'URL complète.
6. Le frontend affiche l'URL et un bouton « Copier ».

Suite : accès anonyme

1. Un tiers reçoit l'URL et l'ouvre.
2. Le composant `PublicView` appelle `GET /api/shares/public/<token>`.
3. L'API détecte qu'un mot de passe est requis et renvoie `{ requirePassword: true }`.
4. Le frontend affiche un formulaire de mot de passe.
5. Le visiteur saisit `Test12345!` et soumet.
6. Nouvel appel avec `?password=Test12345!`, l'API vérifie via `bcrypt` et renvoie les métadonnées du fichier.
7. Le visiteur peut prévisualiser et télécharger.

18.2.4 UC4 – Streaming d'une vidéo

Acteur Utilisateur authentifié

Préconditions

Une vidéo MP4 est uploadée dans son espace.

Scénario nominal

1. L'utilisateur clique sur la vidéo.
2. Le frontend ouvre une modale avec une balise `<video>` dont la source est `/api/files/<id>/stream?token=<jwt>`.
3. Le navigateur émet une requête `GET` avec `Range: bytes=0-`.
4. L'API vérifie le token, ouvre un `createReadStream(path, {start: 0, end: filesize-1})`, renvoie `206 Partial Content` avec `Content-Range: bytes 0-X/total`.
5. L'utilisateur clique pour avancer dans la vidéo (`seek`).
6. Le navigateur réémet `Range: bytes=Y-` où `Y` est la nouvelle position.
7. L'API renvoie une nouvelle portion à partir de `Y`.

18.2.5 UC5 – Téléchargement d'un dossier en ZIP

Acteur Utilisateur authentifié

Préconditions

L'utilisateur a au moins un dossier contenant des fichiers.

Scénario nominal

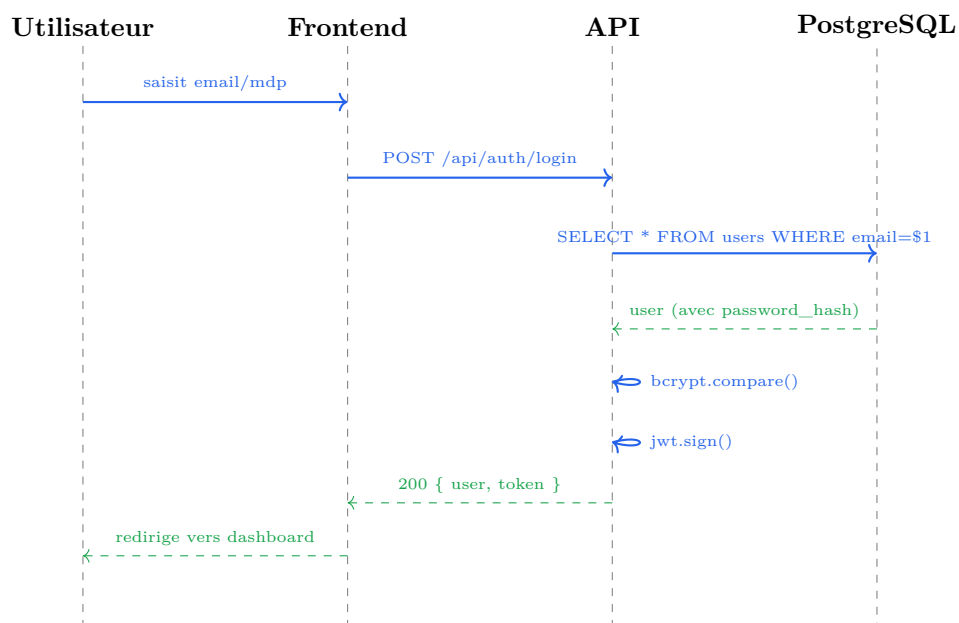
1. L'utilisateur clique sur le menu contextuel d'un dossier, choisit « Télécharger en ZIP ».
2. Le frontend ouvre l'URL `/api/files/folder/<id>/download?token=...` dans un nouvel onglet ou déclenche un download.
3. L'API initialise un `archiver('zip')` dont la sortie est piped vers `res`.
4. L'API parcourt récursivement l'arborescence via `addFolderToArchive` et ajoute chaque fichier physique au flux ZIP.
5. Le navigateur reçoit le ZIP en flux continu pendant qu'il est encore en cours de génération.
6. Une fois terminé, `archive.finalize()` ferme proprement le flux.

Performance

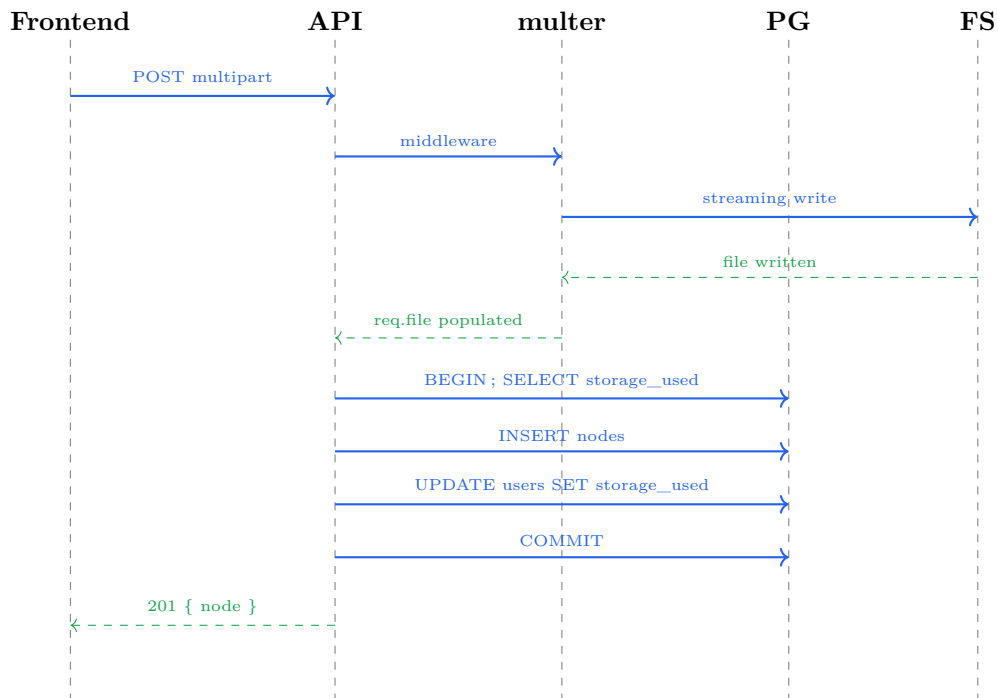
Aucune matérialisation en RAM, possible de descendre des dossiers de plusieurs Go.

18.3 Diagrammes de séquence

18.3.1 Connexion par email/mot de passe



18.3.2 Upload d'un fichier



18.4 Diagrammes UML structurels et comportementaux

18.4.1 Diagramme de classes

Le diagramme de classes présente les entités métier persistées en base de données et leurs relations. Bien que SUPFile n'utilise pas d'ORM (les requêtes SQL sont écrites en SQL natif via pg), la modélisation orientée objet sous-jacente est valide et claire.

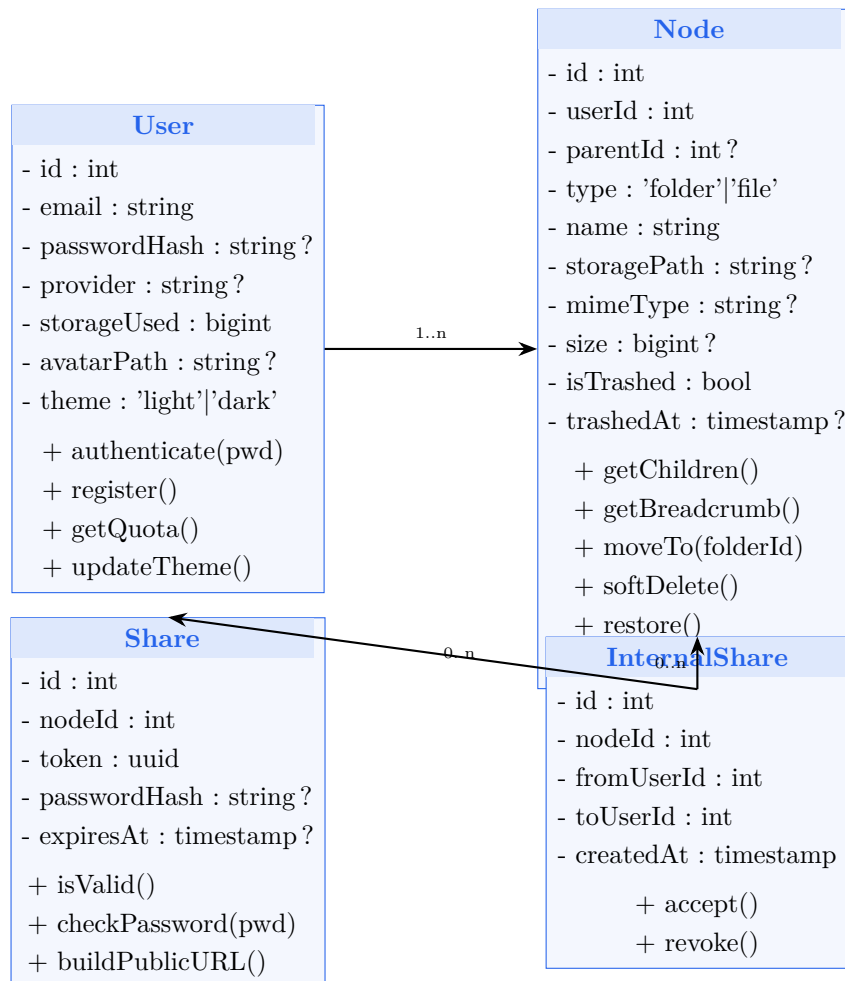


FIGURE 18.2 – Diagramme de classes simplifié des entités métier

18.4.2 Diagramme de déploiement

Le diagramme de déploiement décrit la topologie de l'environnement de production : VPS unique sous Alpine Linux hébergeant la stack Docker derrière Caddy, avec Cloudflare en frontal DNS et Google OAuth pour l'authentification fédérée.

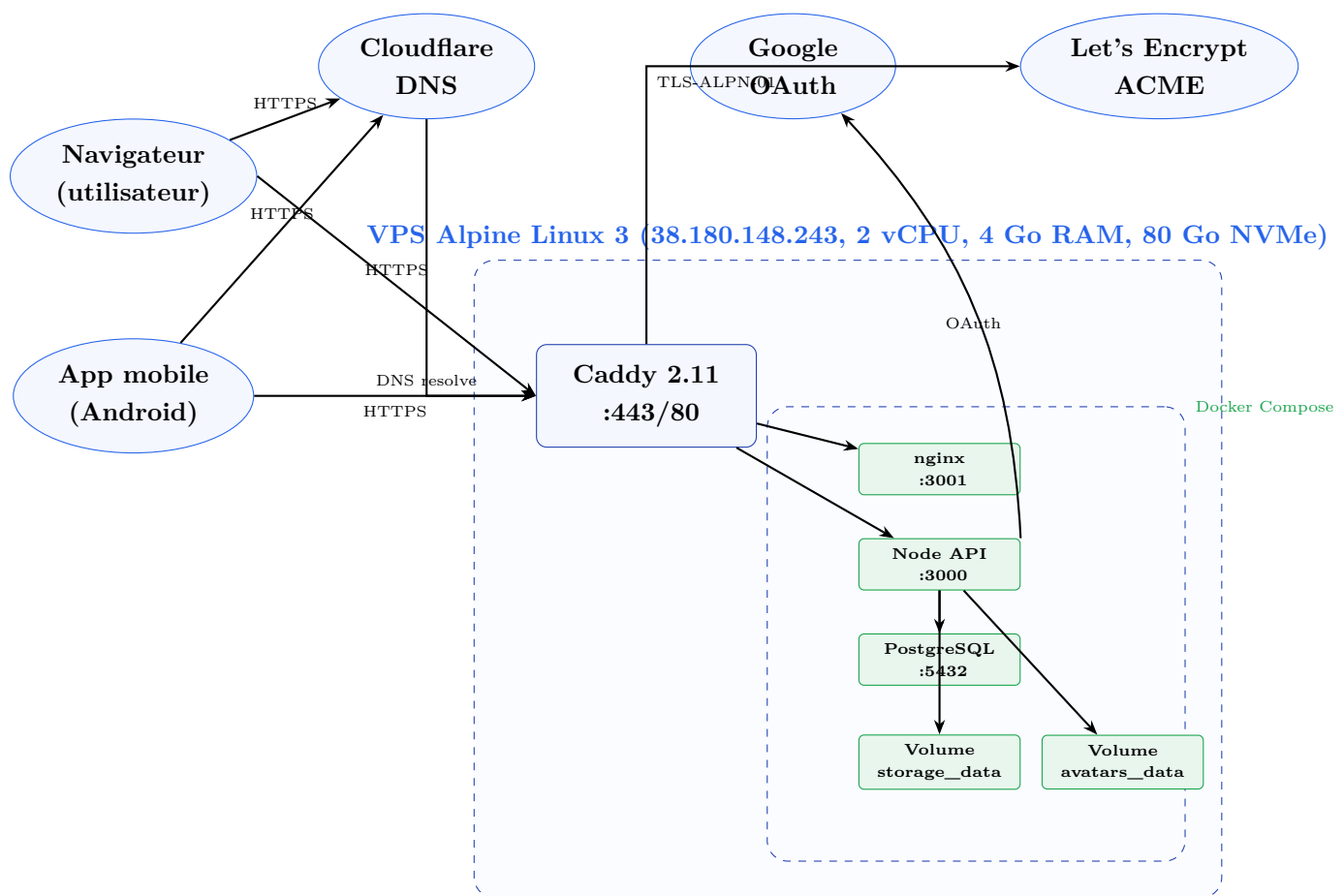


FIGURE 18.3 – Diagramme de déploiement production

18.4.3 Diagramme d'activité : upload d'un fichier

Le workflow d'upload comprend de nombreuses étapes conditionnelles (vérification du quota, gestion d'erreurs, rollback) que le diagramme suivant représente.

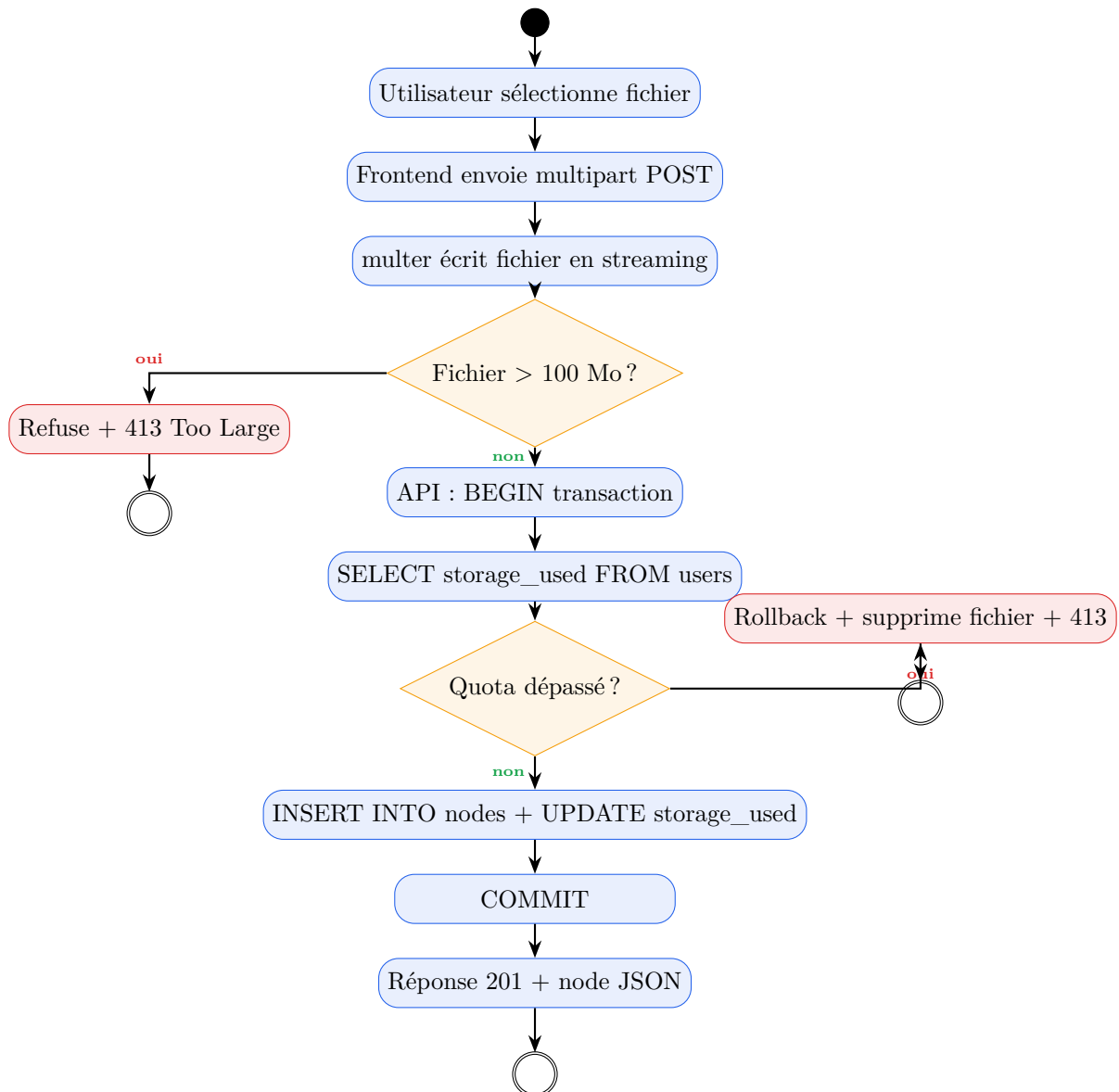


FIGURE 18.4 – Diagramme d'activité : upload d'un fichier

18.4.4 Diagramme d'activité : partage public avec mot de passe

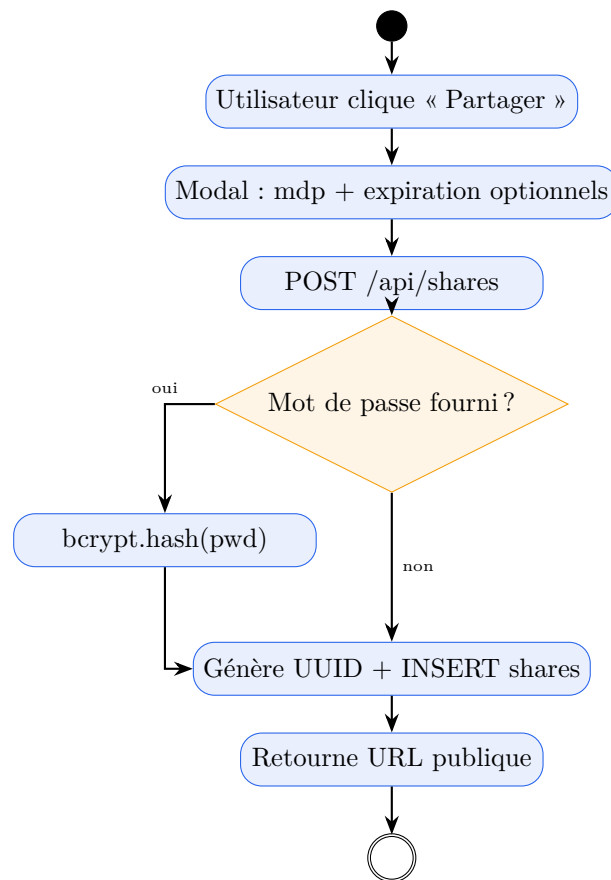


FIGURE 18.5 – Diagramme d'activité : création d'un lien de partage public

18.4.5 Diagramme d'activité : restauration depuis la corbeille

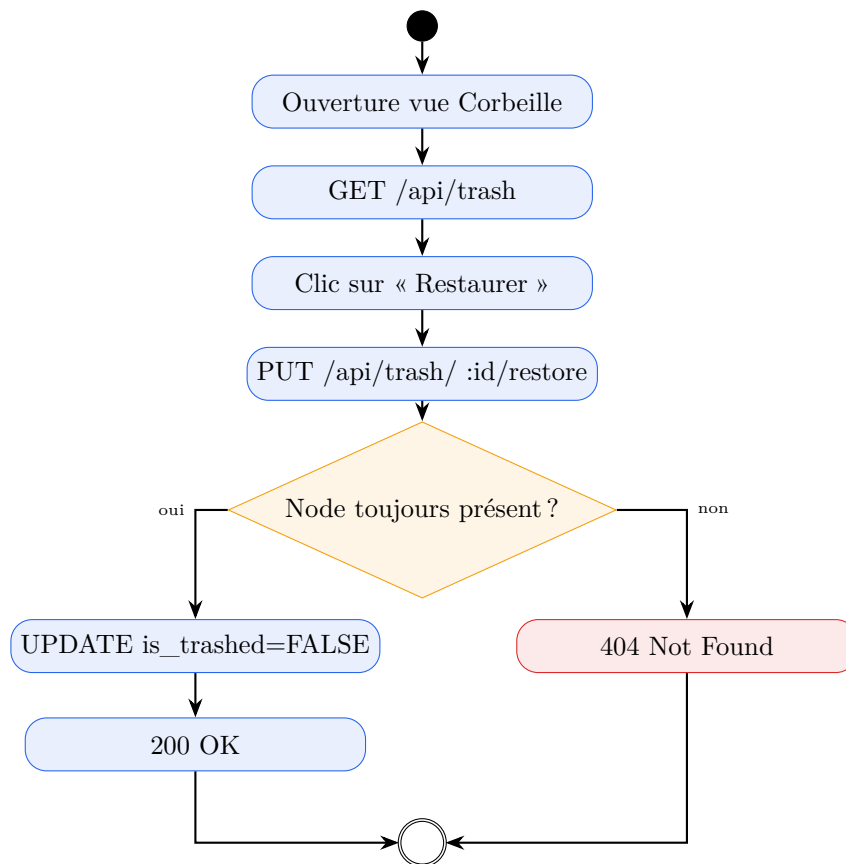


FIGURE 18.6 – Diagramme d'activité : restauration d'un node depuis la corbeille

18.4.6 Diagramme d'états : cycle de vie d'un node

Un fichier ou dossier passe par plusieurs états au cours de sa vie. Le diagramme suivant les présente.

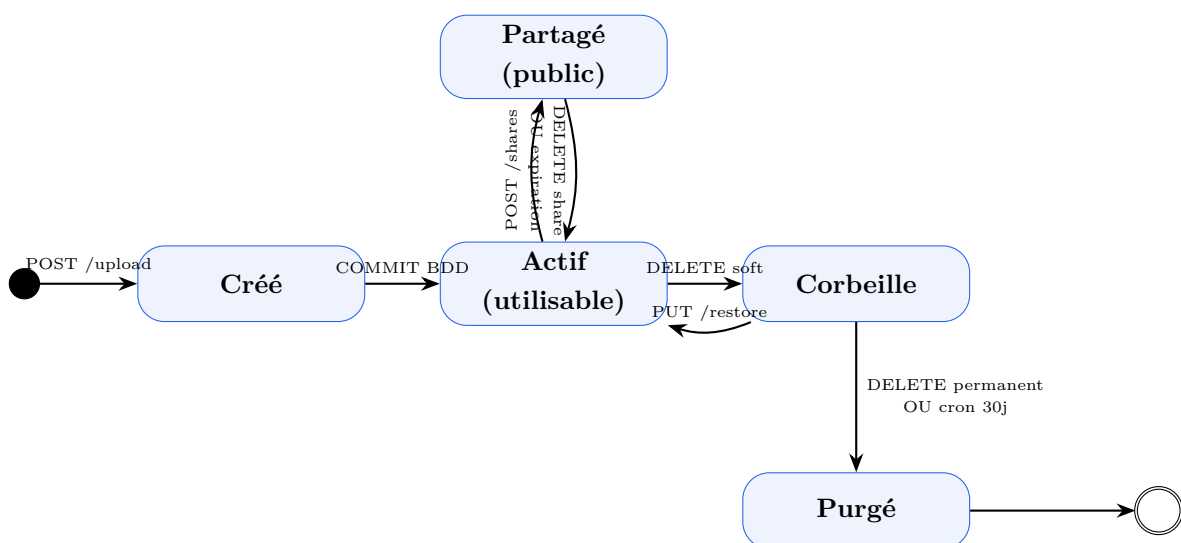
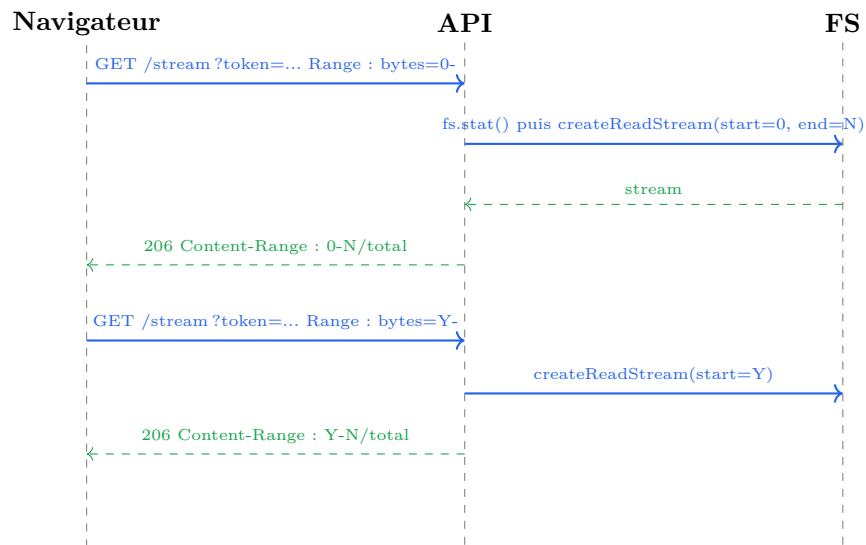


FIGURE 18.7 – Diagramme d'états du cycle de vie d'un node

18.4.7 Lecture d'une vidéo avec Range (séquence)



Chapitre 19

Exemples d'utilisation API (curl)

Cette section présente des exemples concrets d'appels à l'API SUPFile via `curl`, utiles pour tester rapidement les endpoints ou intégrer SUPFile dans des scripts tiers.

19.1 Authentification

19.1.1 Inscription

```
1 curl -X POST https://api.supfile.hackthehydra.com/api/auth/register \
2   -H "Content-Type: application/json" \
3   -d '{
4     "email": "alice@example.com",
5     "password": "Demo12345!"
6   }'
7
8 # Reponse 201
9 {
10  "message": "Compte cree",
11  "user": { "id": 42, "email": "alice@example.com" },
12  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
13 }
```

Listing 19.1 – Inscription d'un nouvel utilisateur

19.1.2 Connexion

```
1 curl -X POST https://api.supfile.hackthehydra.com/api/auth/login \
2   -H "Content-Type: application/json" \
3   -d '{
4     "email": "alice@example.com",
5     "password": "Demo12345!"
6   }'
7
8 # Reponse 200
9 {
10  "user": { "id": 42, "email": "alice@example.com" },
11  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
12 }
13
14 # Stocker le token dans une variable pour la suite
15 TOKEN=$(curl -s -X POST .../api/auth/login -d '...' | jq -r .token)
```

Listing 19.2 – Connexion email/mot de passe

19.1.3 Vérification du token

```
1 curl https://api.supfile.hackthehydra.com/api/users/me \  
2   -H "Authorization: Bearer $TOKEN" \  
3 \  
4 # Reponse 200 \  
5 { \  
6   "id": 42, \  
7   "email": "alice@example.com", \  
8   "avatar_path": null, \  
9   "theme": "dark", \  
10  "provider": null \  
11 }
```

Listing 19.3 – Récupération du profil

19.2 Gestion de l'arborescence

19.2.1 Listing de la racine

```
1 curl "https://api.supfile.hackthehydra.com/api/nodes" \  
2   -H "Authorization: Bearer $TOKEN" \  
3 \  
4 # Reponse 200 \  
5 [ \  
6   { "id": 1, "name": "Documents", "type": "folder", "parent_id": null, ... }, \  
7   { "id": 2, "name": "Photos", "type": "folder", "parent_id": null, ... }, \  
8   { "id": 12, "name": "cv.pdf", "type": "file", "size": 524288, ... } \  
9 ]
```

Listing 19.4 – Liste les nodes de la racine

19.2.2 Listing d'un dossier précis

```
1 curl "https://api.supfile.hackthehydra.com/api/nodes?parent_id=1" \  
2   -H "Authorization: Bearer $TOKEN"
```

Listing 19.5 – Liste les enfants d'un dossier

19.2.3 Création d'un dossier

```
1 curl -X POST https://api.supfile.hackthehydra.com/api/nodes/folder \  
2   -H "Authorization: Bearer $TOKEN" \  
3   -H "Content-Type: application/json" \  
4   -d '{ "name": "Projet 2026", "parent_id": null }' \  
5 \  
6 # Reponse 201
```

```
7 { "id": 99, "name": "Projet 2026", "type": "folder", ... }
```

Listing 19.6 – Création d'un dossier

19.2.4 Breadcrumb

```
1 curl "https://api.supfile.hackthehydra.com/api/nodes/breadcrumb?id=99" \  
2 -H "Authorization: Bearer $TOKEN" \  
3 \  
4 # Reponse 200 \  
5 [ \  
6 { "id": 1, "name": "Documents" }, \  
7 { "id": 50, "name": "Travail" }, \  
8 { "id": 99, "name": "Projet 2026" } \  
9 ]
```

Listing 19.7 – Récupération du fil d'Ariane

19.3 Upload et téléchargement

19.3.1 Upload d'un fichier unique

```
1 curl -X POST https://api.supfile.hackthehydra.com/api/files/upload \  
2 -H "Authorization: Bearer $TOKEN" \  
3 -F "file=@/chemin/local/photo.jpg" \  
4 -F "parent_id=2" \  
5 \  
6 # Reponse 201 \  
7 { \  
8   "id": 123, \  
9   "name": "photo.jpg", \  
10  "type": "file", \  
11  "size": 2097152, \  
12  "mime_type": "image/jpeg", \  
13  "created_at": "2026-05-23T18:42:11Z" \  
14 }
```

Listing 19.8 – Upload d'un fichier

19.3.2 Téléchargement

```
1 curl "https://api.supfile.hackthehydra.com/api/files/123/download" \  
2 -H "Authorization: Bearer $TOKEN" \  
3 --output ./local/photo.jpg
```

Listing 19.9 – Téléchargement d'un fichier

19.3.3 Téléchargement d'un dossier en ZIP

```
1 curl "https://api.supfile.hackthehydra.com/api/files/folder/1/download" \  
2 -H "Authorization: Bearer $TOKEN" \  
3 --output ./local/Documents.zip
```

Listing 19.10 – Téléchargement de dossier en ZIP

19.3.4 Streaming avec Range

```
1 # Recuperation des 1024 premiers octets pour identifier le format  
2 curl "https://api.supfile.hackthehydra.com/api/files/200/stream?token=$TOKEN"  
3 \  
4 -H "Range: bytes=0-1023" \  
5 -i  
6  
7 # HTTP/2 206  
8 # accept-ranges: bytes  
9 # content-range: bytes 0-1023/10485760  
10 # content-length: 1024  
11 # content-type: video/mp4
```

Listing 19.11 – Streaming d'une portion de fichier vidéo

19.4 Partage

19.4.1 Création d'un lien public avec mot de passe

```
1 curl -X POST https://api.supfile.hackthehydra.com/api/shares \  
2 -H "Authorization: Bearer $TOKEN" \  
3 -H "Content-Type: application/json" \  
4 -d '{  
5     "node_id": 123,  
6     "password": "SecretPwd!2026",  
7     "expires_at": "2026-05-30T23:59:59Z"  
8 }'  
9  
10 # Reponse 201  
11 {  
12     "share": {  
13         "id": 7,  
14         "token": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",  
15         "node_id": 123,  
16         "expires_at": "2026-05-30T23:59:59Z"  
17     },  
18     "link": "https://supfile.hackthehydra.com/public/a1b2c3d4-e5f6-7890-abcd-  
19         ef1234567890"
```

Listing 19.12 – Création d'un partage public

19.4.2 Accès anonyme à un lien public

```
1 # Sans mot de passe : detecte que mdp requis
2 curl "https://api.supfile.hackthehydra.com/api/shares/public/a1b2c3d4..."
3
4 # Reponse 200
5 { "requirePassword": true }
6
7 # Avec mot de passe : acces autorise
8 curl "https://api.supfile.hackthehydra.com/api/shares/public/a1b2c3d4...?
9     password=SecretPwd%212026"
10
11 # Reponse 200
12 {
13     "node": { "id": 123, "name": "photo.jpg", "type": "file", ... }
```

Listing 19.13 – Accès anonyme avec mot de passe

19.4.3 Partage interne

```
1 curl -X POST https://api.supfile.hackthehydra.com/api/shares/internal \
2     -H "Authorization: Bearer $TOKEN" \
3     -H "Content-Type: application/json" \
4     -d '{
5         "node_id": 1,
6         "email": "bob@example.com"
7     }'
8
9 # Reponse 201
10 { "id": 12, "node_id": 1, "from_user_id": 42, "to_user_id": 43, ... }
```

Listing 19.14 – Partage avec un utilisateur SUPFile

19.5 Recherche

```
1 curl "https://api.supfile.hackthehydra.com/api/search?q=rapport&type=pdf&date=
2     month" \
3     -H "Authorization: Bearer $TOKEN"
4
5 # Reponse 200
6 [
7     { "id": 88, "name": "rapport_mai.pdf", "size": 1048576, "mime_type": "
8         application/pdf" },
```

```
7 { "id": 89, "name": "rapport_avril.pdf", ... }  
8 ]
```

Listing 19.15 – Recherche par nom et type

19.6 Corbeille

```
1 # Soft delete  
2 curl -X DELETE https://api.supfile.hackthehydra.com/api/nodes/123 \  
3   -H "Authorization: Bearer $TOKEN"  
4  
5 # Listing de la corbeille  
6 curl "https://api.supfile.hackthehydra.com/api/trash" \  
7   -H "Authorization: Bearer $TOKEN"  
8  
9 # Restauration  
10 curl -X PUT https://api.supfile.hackthehydra.com/api/trash/123/restore \  
11   -H "Authorization: Bearer $TOKEN"  
12  
13 # Suppression definitive  
14 curl -X DELETE https://api.supfile.hackthehydra.com/api/trash/123/permanent \  
15   -H "Authorization: Bearer $TOKEN"
```

Listing 19.16 – Workflow corbeille

Monitoring, sauvegarde et exploitation

20.1 Logs applicatifs

L'API émet des logs structurés sur `stdout`, capturés par Docker via le driver de logging par défaut (`json-file`). Pour consulter les logs en production :

```
1 # Logs de l'API en temps reel
2 docker compose -f docker-compose.prod.yml logs -f api
3
4 # 100 dernieres lignes
5 docker compose -f docker-compose.prod.yml logs --tail 100 api
6
7 # Logs de la base de donnees
8 docker compose -f docker-compose.prod.yml logs -f db
9
10 # Logs Caddy (acces HTTPS)
11 tail -f /var/log/caddy/supfile-api.log
12 tail -f /var/log/caddy/supfile-web.log
```

Listing 20.1 – Consultation des logs

Les logs Caddy sont au format JSON et peuvent être facilement parsés via `jq` ou ingérés dans un système ELK / Loki.

20.2 Rotation des logs

Pour éviter la saturation disque, Docker doit être configuré avec une rotation. Créer `/etc/docker/daemon.json`

```
1 {
2   "log-driver": "json-file",
3   "log-opts": {
4     "max-size": "10m",
5     "max-file": "3"
6   }
7 }
```

Listing 20.2 – Configuration de la rotation des logs Docker

Pour Caddy, la rotation est gérée nativement via la directive `log.output file` avec `roll` automatique tous les 100 Mo.

20.3 Métriques système

Pour le monitoring d'infrastructure, plusieurs outils légers sont compatibles :

- **Netdata** : monitoring temps réel installable en une commande.
- **node_exporter + Prometheus + Grafana** : stack standard mais plus lourde.
- **Uptime Kuma** : checks d'uptime HTTP avec alertes.

Pour le VPS SUPFile de démonstration, un simple healthcheck cron suffit en V1 :

```
1 # Cron toutes les 5 minutes
2 */5 * * * * curl -fsS -o /dev/null -w "%{http_code}\n" \
3     https://api.supfile.hackthehydra.com/api/health \
4     >> /var/log/supfile-health.log 2>&1
```

Listing 20.3 – Healthcheck simple

20.4 Backups

20.4.1 Base de données

Stratégie de backup quotidien avec rétention 30 jours :

```
1 #!/bin/sh
2 # /opt/scripts/backup-db.sh
3 BACKUP_DIR=/opt/backups/postgres
4 mkdir -p $BACKUP_DIR
5 DATE=$(date +%Y%m%d-%H%M%S)
6
7 docker compose -f /opt/supfile/docker-compose.prod.yml exec -T db \
8     pg_dump -U supfile supfile | gzip > $BACKUP_DIR/supfile-$DATE.sql.gz
9
10 # Rotation : 30 jours
11 find $BACKUP_DIR -name "supfile-*.sql.gz" -mtime +30 -delete
12
13 # Notification en cas d'echec
14 if [ $? -ne 0 ]; then
15     echo "Backup SUPFile FAILED at $DATE" | mail -s "Backup failed"
16     admin@supfile.fr
17 fi
```

Listing 20.4 – Script de backup

Cron quotidien à 3h du matin :

```
1 0 3 * * * /opt/scripts/backup-db.sh
```

20.4.2 Fichiers utilisateurs

Le volume Docker `storage_data` contient les fichiers physiques. Pour le sauvegarder vers un stockage offsite, `restic` est une excellente solution :

```
1  # Initialisation (une seule fois)
2  restic -r s3:s3.amazonaws.com/supfile-backups init
3
4  # Backup quotidien
5  restic -r s3:s3.amazonaws.com/supfile-backups backup \
6    /var/lib/docker/volumes/supfile_storage_data/_data
7
8  # Snapshot retention
9  restic -r s3:s3.amazonaws.com/supfile-backups forget \
10   --keep-daily 7 --keep-weekly 4 --keep-monthly 6 --prune
```

Listing 20.5 – Backup des fichiers via restic vers S3

20.4.3 Procédure de restauration

En cas de crash du VPS, la restauration suit le scénario suivant :

1. Provisionner un nouveau VPS Alpine.
2. Installer Docker et Caddy (cf. chapitre Déploiement).
3. Cloner le dépôt git.
4. Restaurer la base : `gunzip -c backup.sql.gz | docker compose exec -T db psql -U supfile`
5. Restaurer le volume fichiers : `restic restore latest -target /`
6. Démarrer la stack : `docker compose -f docker-compose.prod.yml up -d`
7. Configurer les DNS si l'IP a changé.

20.5 Gestion des incidents

20.5.1 Symptômes et premiers réflexes

Symptôme	Diagnostic à effectuer
Le site renvoie 502	Vérifier que les conteneurs tournent : <code>docker compose ps</code> . Si arrêtés, <code>docker compose up -d</code> .
HTTPS expiré	Caddy renouvelle auto. Si bloqué, vérifier les logs et que Cloudflare est bien en mode DNS-only.
Quota d'un utilisateur incorrect	Recalculer via <code>SUM(size) WHERE user_id = X AND is_trashed = FALSE</code> .
Upload fail avec 413	Vérifier la limite <code>multer</code> dans <code>api/index.js</code> et les limites de Caddy / nginx amont.
CPU à 100%	Voir si un endpoint particulier explique la charge via les logs API et les metrics PostgreSQL.
Volume disque saturé	<code>docker system prune</code> pour les caches, vérifier la croissance du volume <code>storage_data</code> .

20.5.2 Astreinte et escalade

En production réelle, une astreinte L1 (réponse aux alertes Uptime, redémarrage de conteneurs) doit être organisée avec rotation hebdomadaire. L'escalade L2 (corrections de code) revient à l'équipe de développement.

Chapitre 21

Annexes

21.1 Glossaire

API REST	Application Programming Interface respectant l'architecture REST (Representational State Transfer).
bcrypt	Algorithme de hachage de mots de passe avec salt intégré, conçu pour être lent et résistant aux attaques par force brute.
CTE	Common Table Expression, requête nommée temporaire utilisable dans une autre requête SQL. Particulièrement utile pour les structures récursives.
Docker Compose	Outil de définition et de gestion d'applications Docker multi-conteneurs via un fichier YAML.
HTTP Range	En-tête HTTP <code>Range: bytes=start-end</code> permettant de demander uniquement une portion d'un fichier. Indispensable pour le streaming de médias.
JWT	JSON Web Token, format standardisé de jeton encodé en base64 et signé cryptographiquement.
Let's Encrypt	Autorité de certification gratuite et automatisée fournissant des certificats SSL/TLS pour HTTPS.
Soft delete	Suppression logique consistant à marquer un enregistrement comme supprimé sans le retirer physiquement de la base.

21.2 Références

- Spécification REST : <https://restfulapi.net/>
- Documentation PostgreSQL CTE : <https://www.postgresql.org/docs/current/queries-with.html>
- OWASP Top 10 : <https://owasp.org/www-project-top-ten/>
- Documentation Expo SDK 54 : <https://docs.expo.dev/>
- Documentation Caddy 2 : <https://caddyserver.com/docs/>
- RFC 7235 (HTTP Authentication) : <https://www.rfc-editor.org/rfc/rfc7235>
- RFC 7233 (HTTP Range Requests) : <https://www.rfc-editor.org/rfc/rfc7233>

21.3 Dépôt Git et démo

- Code source : https://github.com/HamerTails/4_Proj_Drive
- Démo web : <https://supfile.hackthehydra.com>
- API publique : <https://api.supfile.hackthehydra.com/api/health>

21.4 Variables d’environnement

Variable	Côté	Description
POSTGRES_DB	db, api	Nom de la base (par défaut <code>supfile</code>)
POSTGRES_USER	db, api	Utilisateur PG (par défaut <code>supfile</code>)
POSTGRES_PASSWORD	db, api	Mot de passe PG (généré via <code>openssl rand -base64 24</code>)
DATABASE_URL	api	URL complète PG (<code>postgres://user:pwd@db:5432/supfile</code>)
JWT_SECRET	api	Secret de signature des JWT (<code>openssl rand -hex 32</code>)
PORT	api	Port d’écoute (par défaut 3000)
NODE_ENV	api	<code>development</code> ou <code>production</code>
WEB_URL	api	URL du frontend web pour CSP et CORS
GOOGLE_CLIENT_ID	api	Identifiant OAuth Google
GOOGLE_CLIENT_SECRET	api	Secret OAuth Google
OAUTH_CALLBACK_URL	api	URL de callback (<code>http://localhost:3000/api/auth/google/callback</code> en dev)
STORAGE_QUOTA_BYTES	api	Quota par utilisateur en octets (par défaut 30×1024^3)
CORS_ORIGINS	api	Liste séparée par virgules des origines autorisées
RATE_LIMIT_AUTH	api	Nombre de tentatives auth/min (5 en prod, 100 en dev)
VITE_API_URL	web	URL de l’API utilisée par le frontend web
EXPO_PUBLIC_API_URL	mobile	URL de l’API embarquée à la build APK

21.5 Arborescence du dépôt

```

1 4PROJ/
2 |-- api/                                # Backend Node.js

```

```

3 | | -- index.js # Point d'entree, middlewares globaux
4 | | -- db.js # Connexion pg.Pool + schema/indexes
5 | | -- middleware/
6 | | | -- auth.js # Verification JWT
7 | | | -- validate.js # Schemas de validation
8 | | -- routes/
9 | | | -- auth_logic.js # /api/auth/*
10 | | | -- nodes.js # /api/nodes/*
11 | | | -- files.js # /api/files/*
12 | | | -- trash.js # /api/trash/*
13 | | | -- storage.js # /api/storage/*
14 | | | -- shares.js # /api/shares/*
15 | | | -- search.js # /api/search
16 | | | -- users.js # /api/users/*
17 | | -- utils/
18 | | | -- mime.js # Helpers MIME
19 | | -- Dockerfile
20 | | -- package.json
21 |
22 | -- web/ # Frontend React + Vite
23 | | -- src/
24 | | | -- App.jsx
25 | | | -- main.jsx
26 | | | -- index.css # Variables CSS theme
27 | | | -- components/
28 | | | | -- Login.jsx
29 | | | | -- Register.jsx
30 | | | | -- Dashboard.jsx # Navigateur de fichiers (~2400 lignes)
31 | | | | -- DashboardHome.jsx # Accueil avec quota
32 | | | | -- Settings.jsx
33 | | | | -- Trash.jsx
34 | | | | -- PublicView.jsx
35 | | | | -- Toast.jsx
36 | | | | -- Skeleton.jsx
37 | | | -- services/
38 | | | | -- api.js # Client HTTP centralise
39 | | -- public/
40 | | -- Dockerfile
41 | | -- nginx.conf
42 | | -- vite.config.js
43 | | -- package.json
44 |
45 | -- mobile/ # App React Native + Expo
46 | | -- app/ # Routes (expo-router)
47 | | | -- _layout.tsx
48 | | | -- index.tsx # Accueil
49 | | | -- login.tsx
50 | | | -- register.tsx
51 | | | -- share.tsx
52 | | | -- (tabs)/
53 | | | | -- _layout.tsx

```

```

54 | | | | -- files.tsx          # Navigateur de fichiers
55 | | | | -- dashboard.tsx
56 | | | | -- shared.tsx
57 | | | | -- trash.tsx
58 | | | | -- settings.tsx
59 | | -- components/
60 | | -- contexts/          # ThemeContext, etc.
61 | | -- services/         # authService, filesService, ...
62 | | -- constants/
63 | | | | -- api.ts        # Axios instance + interceptors
64 | | -- assets/
65 | | -- app.json
66 | | -- eas.json
67 | | -- package.json
68 |
69 | -- docs/                # Documentation
70 | | -- DOCS.md           # Doc technique condensee
71 | | -- USER_MANUAL.md
72 | | -- API.md
73 | | -- UML.md
74 | | -- MOBILE.md
75 | | -- WEB.md
76 | | -- technical_documentation.tex # Ce document
77 |
78 | -- scripts/            # Scripts ops
79 | | -- build_rendu_zip.sh
80 |
81 | -- docker-compose.yml   # Dev local
82 | -- docker-compose.prod.yml # Production
83 | -- Caddyfile           # Reverse proxy prod
84 | -- .env.example
85 | -- .gitignore
86 | -- README.md
87 | -- LICENSE

```

Listing 21.1 – Structure principale du projet

21.6 Dépendances principales

21.6.1 Backend (api/package.json)

Paquet	Version	Rôle
express	^5.0	Framework HTTP
pg	^8.13	Driver PostgreSQL
bcrypt	^5.1	Hash de mots de passe
jsonwebtoken	^9.0	Génération et vérification JWT
multer	^1.4	Upload multipart
archiver	^7.0	Génération ZIP en streaming
helmet	^8.0	En-têtes HTTP de sécurité
cors	^2.8	Gestion CORS
express-rate-limit	^7.5	Rate limiting
passport	^0.7	Auth multi-provider
passport-google-oauth20	^2.0	Stratégie Google
dotenv	^16.4	Chargement .env

21.6.2 Frontend web (web/package.json)

Paquet	Version	Rôle
react	^18.3	Bibliothèque UI
react-dom	^18.3	Rendu DOM
react-router-dom	^6.28	Routing SPA
recharts	^2.13	Graphiques (camembert quota)
vite	^5.4	Bundler de dev et build
@vitejs/plugin-react	^4.3	Plugin React pour Vite

21.6.3 Application mobile (mobile/package.json)

Paquet	Version	Rôle
expo	~54.0	SDK Expo
expo-router	~6.0	Routing fichier
react	19.1	UI
react-native	0.81	Native bridge
react-native-safe-area-context	5.6	Safe areas
@react-native-async-storage/async-storage	2.2	Storage local
expo-image	~3.0	Image optimisée
expo-image-picker	~17.0	Picker caméra/galerie
expo-document-picker	~14.0	Picker documents
expo-av	~16.0	Audio/vidéo
expo-web-browser	~15.0	Browser OAuth
axios	^1.7	HTTP client

21.7 Conformité et licences

21.7.1 Licence du projet

Le code source de SUPFile est publié sous licence **MIT**, autorisant l'usage commercial, la modification, la distribution et l'utilisation privée. Le fichier **LICENSE** à la racine du dépôt contient le texte complet.

21.7.2 Licences des dépendances

Toutes les dépendances utilisées sont sous des licences permissives compatibles MIT : MIT, Apache 2.0, BSD-2-Clause, BSD-3-Clause, ISC. Aucune dépendance sous licence copyleft (GPL, AGPL) n'est utilisée, ce qui garantit que SUPFile peut être intégré dans des produits commerciaux propriétaires.

21.7.3 RGPD

Le projet respecte les principes du RGPD :

- **Minimisation** : seules les données strictement nécessaires sont collectées (email, mot de passe hashé, fichiers uploadés).
- **Consentement** : l'inscription explicite à un compte vaut consentement aux Conditions Générales (à rédiger en V2).
- **Droit à l'oubli** : la suppression de compte (`DELETE /api/users/account`) efface intégralement et irréversiblement toutes les données associées (cascade).
- **Portabilité** : le téléchargement ZIP de la racine permet d'exporter l'intégralité de ses fichiers en une opération.
- **Sécurité** : chiffrement HTTPS de bout en bout, mots de passe hashés avec bcrypt, prepared statements anti-injection.
- **Localisation** : hébergement en France/UE (en V1 sur VPS européen, en V2 sur l'infrastructure SUPFile interne).

21.7.4 Audit de sécurité préliminaire

Un audit interne a été conduit en mai 2026 sur les points OWASP Top 10 :

Catégorie OWASP	Statut V1	Notes
A01 :2021 Broken Access Control	OK	Vérification d'ownership systématique
A02 :2021 Cryptographic Failures	OK	bcrypt + JWT + HTTPS
A03 :2021 Injection	OK	Prepared statements partout
A04 :2021 Insecure Design	OK	Pas de fonctionnalité connue exposée à risque
A05 :2021 Security Misconfiguration	OK	helmet, CSP stricte, rate limit
A06 :2021 Vulnerable Components	À surveiller	Audit npm audit régulier
A07 :2021 Identification & Auth Failures	OK	Rate limit, hash robuste
A08 :2021 Software & Data Integrity	OK	Pas de chargement de code dynamique
A09 :2021 Security Logging	Partiel	Logs OK mais pas de SIEM en V1
A10 :2021 SSRF	OK	Pas de proxy ou fetch côté serveur

Un pentest externe complet est planifié pour la V2 avant publication officielle.

21.8 Captures d'écran de référence

Les captures suivantes peuvent être insérées dans cette section pour documenter visuellement l'application. Elles sont disponibles dans le dépôt `docs/screenshots/` après exécution du parcours utilisateur de test (71 captures générées).

21.8.1 Frontend web

- `01_landing.png` : page d'accueil non authentifiée.
- `02_register.png` : formulaire d'inscription.
- `03_login.png` : formulaire de connexion.
- `04_dashboard_empty.png` : dashboard juste après inscription (quota vide).
- `05_upload_dragdrop.png` : zone d'upload pendant un drag and drop.
- `06_files_grid.png` : navigateur de fichiers en grille.
- `07_files_list.png` : navigateur en liste.
- `08_preview_image.png` : prévisualisation d'image.
- `09_preview_pdf.png` : prévisualisation PDF en iframe.

- 10_preview_video.png : streaming vidéo.
- 11_share_modal.png : modale de création de partage.
- 12_public_view.png : vue publique d'un lien partagé.
- 13_trash.png : corbeille.
- 14_settings.png : paramètres utilisateur.
- 15_dark_mode.png : interface en thème sombre.

21.8.2 Application mobile

- m01_splash.png : écran de splash au lancement.
- m02_landing.png : écran d'accueil avec choix Google / Email.
- m03_login.png : formulaire de connexion.
- m04_files.png : navigateur de fichiers.
- m05_upload_menu.png : modal de choix de source d'upload.
- m06_camera.png : capture depuis l'appareil photo.
- m07_dashboard.png : dashboard avec quota.
- m08_share.png : écran de partage.
- m09_trash.png : corbeille mobile.
- m10_settings.png : paramètres mobile.

21.9 Procédure de compilation de ce document

Ce document LaTeX peut être compilé localement avec une distribution TeX Live ou MiKTeX, ou via Overleaf en ligne.

21.9.1 Compilation locale

```
1  # Installation TeX Live (Debian/Ubuntu)
2  sudo apt install texlive-full
3
4  # Compilation (3 passes pour TOC et references)
5  cd docs/
6  pdflatex technical_documentation.tex
7  pdflatex technical_documentation.tex
8  pdflatex technical_documentation.tex
9
10 # Le PDF est genere : technical_documentation.pdf
```

Listing 21.2 – Compilation locale du PDF

21.9.2 Compilation via Overleaf

1. Créer un compte gratuit sur <https://www.overleaf.com>.
2. Cliquer sur « New Project » → « Upload Project ».
3. Uploader le dossier `docs/` compressé en ZIP.
4. Sélectionner `technical_documentation.tex` comme document principal.
5. Lancer la compilation (bouton « Recompile »).

21.9.3 Personnalisation

Pour adapter le document (changer les couleurs, ajouter un logo, modifier l’auteur) :

- Couleurs principales : variables `\definecolor` en début de fichier.
- Polices : remplacer `\usepackage{lmodern}` par un autre package (par exemple `\usepackage{libertine}`).
- Logo de page de garde : remplacer le bloc `tikzpicture` par `\includegraphics[width=4cm]{logo.png}`.
- En-têtes : modifier les directives `\fancyhead` et `\fancyfoot`.

21.10 Critères d’acceptation par fonctionnalité

Chaque fonctionnalité majeure dispose d’une définition de « prêt » (Definition of Done) utilisée comme grille de validation par l’équipe.

21.10.1 Authentification

- L’utilisateur peut créer un compte avec email et mot de passe (≥ 6 caractères).
- Un email invalide est refusé côté API et côté UI avec message explicite.
- Le mot de passe est hashé en base avec `bcrypt` (10 rounds).
- Le JWT est généré et persisté côté client (`localStorage` / `AsyncStorage`).
- L’utilisateur authentifié est redirigé vers le dashboard.
- L’utilisateur peut se déconnecter, le token est supprimé.
- Un compte Google peut être créé en un clic via `OAuth`, sans saisie de mot de passe.

21.10.2 Upload

- L’upload accepte tous types MIME jusqu’à 100 Mo.
- La barre de progression s’affiche pendant l’upload.
- Le quota est vérifié avant insertion en BDD (transaction atomique).
- En cas de dépassement, le fichier physique est supprimé et l’utilisateur reçoit un toast d’erreur explicite.
- Le fichier apparaît immédiatement dans la liste après succès.

21.10.3 Partage public

- Le lien généré est un UUID v4, donc imprévisible.
- L'URL est copiable en un clic via le bouton dédié.
- Un mot de passe optionnel est hashé avec bcrypt avant stockage.
- Une date d'expiration optionnelle est vérifiée à chaque accès.
- L'accès anonyme fonctionne sans cookie de session.
- L'expiration ou la suppression du share rend immédiatement le lien invalide.

21.10.4 Recherche

- La recherche est insensible à la casse et aux accents.
- Au moins un critère doit être fourni (q, type, ou date).
- Les filtres type et date sont combinables avec le terme.
- Le temps de réponse est inférieur à 500 ms sur 10 000 nodes.

21.11 SLA et engagement de niveau de service

Pour un déploiement professionnel, SUPFile s'engage sur les SLA suivants :

Indicateur	Cible	Mesure
Disponibilité mensuelle	99.5%	Uptime via Uptime Kuma + Healthcheck
Temps de réponse P95 API	< 500 ms	Métrique Caddy logs
Temps de réponse P99 API	< 1.5 s	Idem
Durée d'un upload 10 Mo	< 5 s	Test mensuel
RTO (Recovery Time Objective)	< 1h	Bascule sur VPS de secours
RPO (Recovery Point Objective)	< 24h	Backup quotidien automatisé
Temps de support critique	4h ouvrées	Email + escalade téléphonique
Notification incident	< 30 min	Status page + email

21.12 Plan de capacité

21.12.1 Capacité actuelle (V1 VPS 2 vCPU / 4 Go RAM)

- Concurrence supportée : ~ 100 connexions simultanées (Node mono-thread + PostgreSQL).

- Utilisateurs actifs estimés : 500 à 1 000 sans dégradation perceptible.
- Stockage : 80 Go disponibles $\div$ 30 Go quota moyen $\approx$ 2-3 utilisateurs intensifs ou 50 utilisateurs faiblement actifs (estimation $\sim$ 1 Go moyenne).
- Bande passante : 1 Gbps en up et down sur l'interface VPS.

21.12.2 Seuils d'alerte

- Disque $> 80\%$: alerte mail $\rightarrow$ provisionner volume additionnel.
- RAM $> 90\%$: analyse logs $\rightarrow$ scaler vCPU ou optimiser requêtes.
- CPU $> 85\%$ sur 10 min : alerte $\rightarrow$ scaler ou identifier endpoint coupable.
- Latence API > 1 s P95 : alerte $\rightarrow$ analyse requêtes lentes (`pg_stat_statements`).

21.12.3 Plan de scaling

1. **Phase 1 (jusqu'à 5 000 users)** : scaling vertical du VPS (4 puis 8 vCPU, 16 Go RAM).
2. **Phase 2 (5 000 à 50 000 users)** : séparation des services : VPS API dédié + VPS BDD dédié + bucket S3 pour fichiers.
3. **Phase 3 (50 000 à 500 000 users)** : Kubernetes managé (par exemple OVH MKS), réplication PostgreSQL (1 master, 2 replicas read), CDN Cloudflare pour fichiers publics.
4. **Phase 4 (500 000+ users)** : sharding BDD par tenant, multi-région.

21.13 Contact et support

Pour toute question technique ou demande d'évolution :

- Issues GitHub : https://github.com/HamerTails/4_Proj_Drive/issues
- Email équipe : `contact@supfile.fr` (placeholder, à configurer en V2)
- Démo et tests : <https://supfile.hackthehydra.com> (comptes `alice.demo@supfile.test` et `bob.demo@supfile.test`, mot de passe `Demo12345!`)

Fin de la documentation technique SUPFile V1.0

Mai 2026 – Projet 4PROJ SUPINFO